高级语言程序设计

第2章：控制语句

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2024/2/8

- ◆2.1 程序块、循环和分支
- ◆2.2 程序设计中的算法
- ◆2.3 Java语言中的异常处理简介
- ◆2.4 数组

2.1.1 程序块

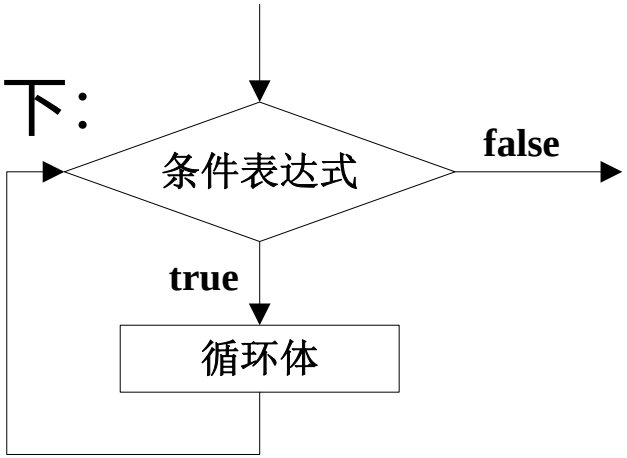
- 又叫复合语句，语句块。用{ }把一些语句括起来构成复合语句。
- 在可以出现语句的任何地方都可以合法地使用程序块。
- Java语言的复合语句与C++复合语句不同的是：Java语言不允许在两个嵌套的复合语句内声明两个同名的变量。

```
public class E0201DeclarBlockVariables {
    public static void main(String[] args) {
        //int k=0; 这个k和下面嵌套的语句块内k同名，会引起编译错误。
        {
            int k=5;
            System.out.print(k);
        }
        { //这个语句块和上面语句块并列，同名变量k没有问题。
            int k=6;
            System.out.println(","+k);
        }
    }
}
```

◆while语句

■实现“当型”循环，其一般语法格式如下：

```
while (check-expression) {
    //body of the loop;
    <statements>
}
```



■解释

- 条件表达式(check-expression)的返回值为布尔型
- 循环体可以是单个语句，也可以是复合语句块

■执行过程

- 先判断check-expression的值，为真则执行循环体
- 循环体执行完后再无条件转向条件表达式做计算与判断；当计算出条件表达式的值为假时，跳过循环体执行while语句后面的语句。若为真，则继续执行循环

2.1.2 while循环：

◆举例：求1-100的和

```
public class E0202WhileDemoSum {
    public static void main(String[] args) {
        // 使用while循环求1~100的和
        int sum = 0;
        int i = 1;
        while (i <= 100) {
            sum += i;
            i++;
        }
        System.out.println("1~100的和是： " +
sum);
    }
}
//输出结果：
//1~100的和是： 5050
```

2.1.2 while循环

◆运行方法

javac WhileDemoSum.java（javac <java文件名>）

java WhileDemoSum （java <类名>）

或者：

java WhileDemoSum.java (Java 11新特性。不生成字节码直接执行，便于快速测试小型程序)



◆运行结果会发现，中文可能变成了乱字符。

- 这是因为源代码保存的编码格式是UTF-8, 不是GBK. 为了解决这个问题，要么转换为GBK格式，要么追加命令行参数，指明文件编码方式。

java -Dfile.encoding=UTF-8 WhileDemoSum.java

或者

javac -encoding UTF-8 WhileDemoSum.java

java WhileDemoSum

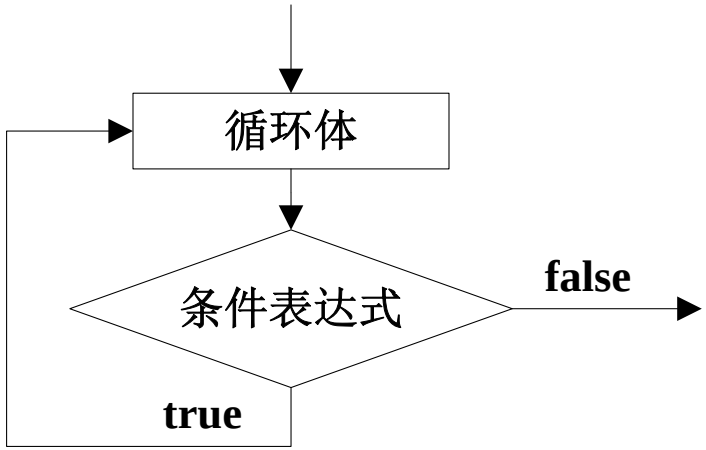
2.1.3 do...while语句

◆do-while语句

- ■实现“直到型”循环
- ■一般语法结构如下

```
do {
    //body of the loop;
} while (check-expression);
```

- ■其使用与while语句很类似，不同的是它首先无条件的执行一遍循环体，再来判断条件表达式的值，若表达式的值为真，则再运行循环体，否则跳出do-while循环，执行下面的语句
- ■特点：它的循环体至少要执行一次



2.1.3 do...while语句

◆比如用户游戏时，会先游戏一次，然后系统提问是否要继续玩。如果用户选择'yes', 则进行下一次游戏。这种情况用do...while循环实现更简便。

```
do {
    Play a Game
    Ask user if he wants to play another game
    Read the user's response
} while ( the user's response is yes );
```

◆这个伪代码实现为Java代码如下:

2.1.3 do...while语句：


```
import java.util.Scanner;
public class E0203WhileDemoPlayGame {
  public static void main(String[] args){
    Scanner scanin = new Scanner(System.in); //scanner for input
    boolean wantsToContinue; // 为真(true)时继续游戏
    do {
        Checkers.playGame(); //函数调用
        System.out.print("Do you want to play again? "); //提示用户输入
        String ans = scanin.nextLine(); //等待yes或者no
        wantsToContinue = ans.equalsIgnoreCase("yes");
    } while (wantsToContinue == true);
    System.out.println("不玩了。");
    scanin.close();
  }
}
class Checkers {
  public static void playGame(){
    System.out.println("游戏中。。。");
    System.out.println("游戏结束。");
  }
}
```

2.1.3 do...while语句

◆代码说明

- 一个Java文件中可以放多个class. 但是只有一个class可以是public, 而且必须和文件同名（包括大小写）
- 像wantsToContinue这样用来控制是否继续循环的变量，叫做标志变量(flag).
- while (wantsToContinue==true)也可以写作while(wantsToContinue)，这样更直观，接近人的自然阅读。
- equalsIgnoreCase是字符串类的一个方法，忽略大小写比较字符串否相等。
- 程序结束前用scanin.close()关闭打开的scanner. 虽然程序退出系统会自动关闭并释放资源，但是在代码中及时关闭不再使用的文件或者scanner是很重要的。

2.1.3 do...while语句

◆任何可以使用 do..while循环解决的程序逻辑也可以使用 while 语句来解决，反之亦然。

```
do {
    <doSomething>
} while (<boolean-expression>);
```

与下面代码效果等价：

```
<doSomething>
while (<boolean-expression>) {
    <doSomething>
}
```

```
while (<boolean-expression>) {
    <doSomething>
}
```

与下面代码效果等价：

```
if (<boolean-expression>) {
  do {
    <doSomething>
  } while (<boolean-expression>);
}
```

2.1.3 do...while语句

◆举例：求1-100的和

```
public class E0204DoWhileDemoSum {
    public static void main(String[] args) {
        // 使用do-while循环求1~100的和
        int sum = 0;
        int i = 1;
        do {
            sum += i;
            i++;
        } while (i <= 100);
        System.out.println("1~100的和是：" +
sum);
    }
}
//输出结果:
//1~100的和是：5050
```

break语句

- 使程序的流程从一个语句块（switch或循环结构）内跳出。
- break [标号];

用于指定一个封闭的语句块，可以省略

continue语句

- 终止当前这一轮（次）的循环，进入下一轮（次）循环。
- continue [标号];

应为定义在程序中某一循环语句的前面，用来标志该循环，它使得程序直接转入标号标明的循环层次。可以省略

2.1.4 break语句和continue语句

◆例:使用了continue和break语句,计算1+3+5...输出100内的素数.

```
public class E0206BreakContinueDemo {
  public static void main(String args[]) {
    int sum=0,i,j;
    for( i=1;i<=10;i++) {
      if(i%2==0) { //计算1+3+5+7+9
        continue;
      }
      sum=sum+i;
    }
    System.out.println("sum="+sum);
    for(j=2;j<=100;j++) { //求100以内的素数
      for( i=2;i<=j/2;i++) {
        if(j%i==0) break;
      }
      if(i>j/2) {
        System.out.println(""+j+"是素数");
      }
    } }}
```

2.1.4 break语句和continue语句

◆带标号的break和continue举例

```
1   loop:while (true){
2           for( … ){
3              switch( ){
4                 case -1:
5                 case n:
6                    break loop ; //跳出while去11行
7                                 …
8                 }
9           }
10        }
11  test:for( … ){
12          …
13          while(… ){
14            if( …){
15              …
16              continue test ; //到11行
17            }
18          }
19        }
```

2.1.5 for语句

◆for语句的语法格式：

```
for (start-expression; check-expression; update-expression) {
    //body of the loop;
}
```

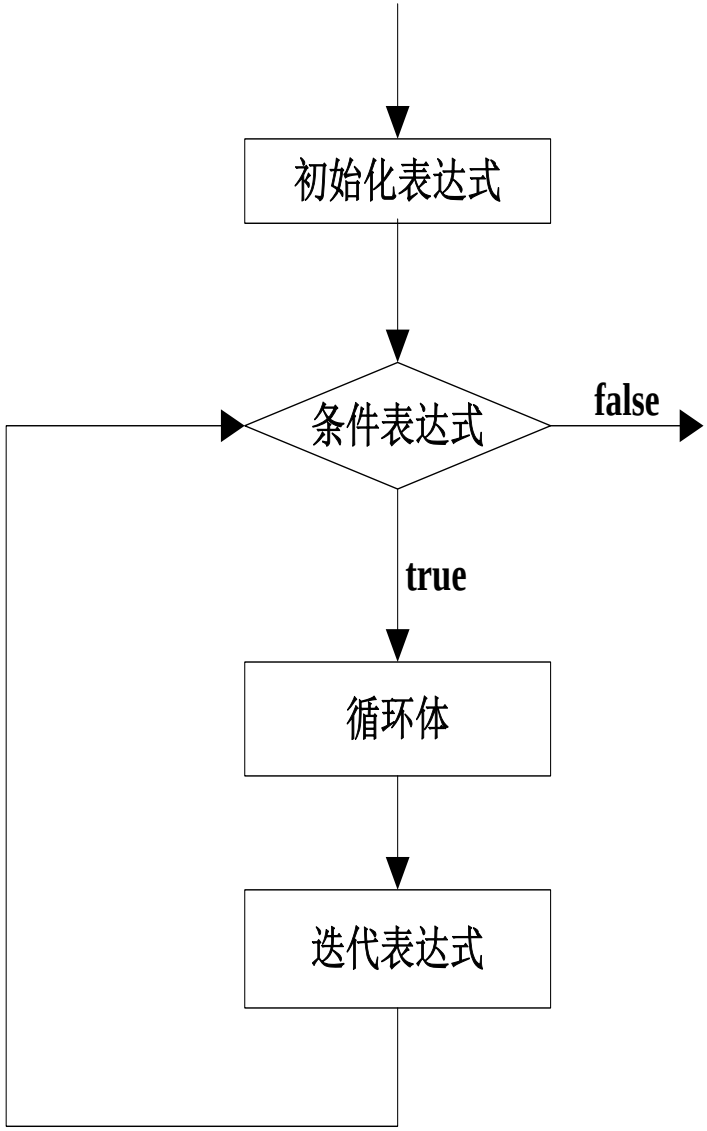
◆for语句的执行规则是：

(1)计算“start-expression”，完成必要的初始化工作。

(2)判断“check-expression”的值，若“check-expression”的值为true，则进行(3),否则进行(4)。

(3)执行循环体，然后计算“update-expression”，以便改变循环条件，进行（2）。

(4)结束for语句的执行。



◆for举例：打印九九乘法表

```
public class E0207ForDemoMultiTable {
    public static void main(String[] args){
        for (int i=1; i<=9;i++) {
            for (int j=1; j<=i;j++)
                System.out.print("  "+i+"*"+j+"="+i*
j);
            System.out.println();
        }
    }
 }
```

2.1.5 for语句

◆增强for循环

- Java 5的新特色. 又叫foreach循环。
- 用来对数组或者集合对象进行遍历
- 语法格式如下:
- 其中Type必须和数组或者集合的元素类型一致。

```
for (Type item : Array/Set) {
    //body of the loop;
}
```

◆Foreach举例：用两种方式遍历数组。

```
public class E0208ForeachDemo {
  public static void main(String args[]) {
    int a[] = {1,2,3,4};
    String b[] = {"Java","C#","Python"};
    for(int n=0;n<a.length;n++) { //传统方式
      System.out.println(a[n]);
    }
    for(int n=0;n<b.length;n++) { //传统方式
      System.out.println(b[n]);
    }
    for(int i:a) {//循环变量i依次取数组a的每一个元素的值
          System.out.println(i);
    }
    for(String str:b) { //循环变量ch依次取数组b的每一个元素的值
         System.out.println(str);
    }      } }
```

2.1.5 for语句: 类型推断**

- Java 10新特性. 根据局部变量的初始化值，编译器推断出变量类型。从而简化代码书写(语法糖)
- 借鉴自C#和JavaScript中的var

适用场合

- 1.局部变量初始化
- 2. for循环内部索引变量
- 3. 传统的for循环声明变量

不适用场合

- 1.方法参数
- 2.构造函数参数
- 3.方法返回类型
- 4.类成员变量类型

2.1.5 for语句: 类型推断**

◆类型推断举例:

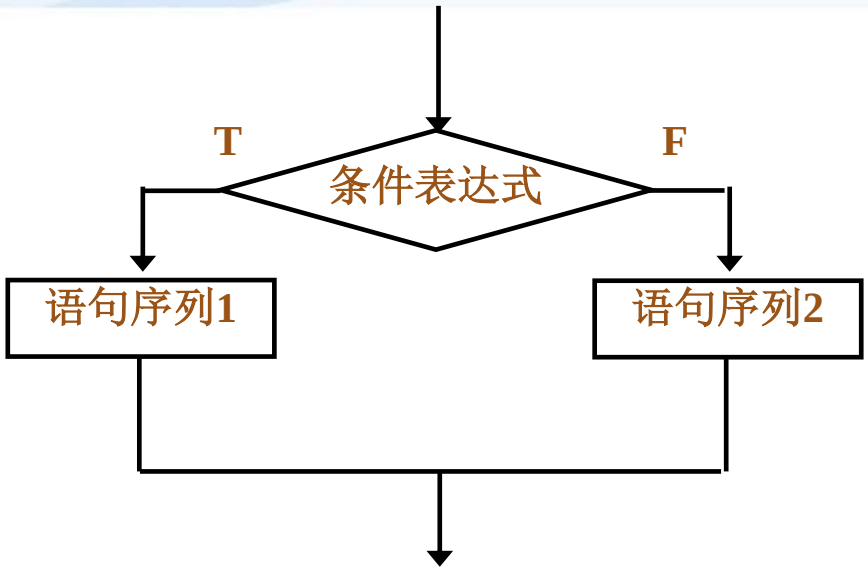
```
public class E0209VarDemo{
   public static void main(String[] args)
   {
        var hello = "Hello, Java";
        int[] arr = {0,1,2};
        System.out.println(hello);
        for (var i: arr) {
            System.out.println("arr i="+i);
        }
        for (var i=1; i<5; i++)
        {
            System.out.println("i="+i);
        }
   }
}
```

输出结果

```
Hello, Java
arr i=0
arr i=1
arr i=2
i=1
i=2
i=3
i=4
```

if双路条件结构

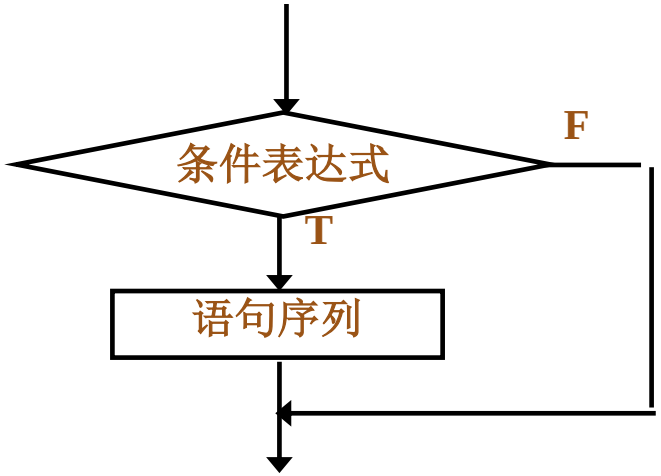
```
if (条件表达式)
{
  语句序列 1
}
else
{
  语句序列 2
}
```



双路条件选择结构

if单路条件结构

```
if  (条件表达式)
 {
    语句序列
}
```



单路条件选择结构

◆举例：求三个a,b,c中的最大值和最小值。

```
public class E0210IfelseDemo {
  public static void main(String[] args)
  {
    int a=100,b=20,c=300,max,min;
    if(a>b) { //用if-else求最大值
          max=a;
    } else {
      max=b;
    }
    if(c>max) max=c;
    System.out.println("Max="+max);
    min=a<b ? a : b; //用表达式求最小值
    min=c<min ? c : min;
    System.out.println("Min="+min);
  }
}
```

2.1.7 if语句的完备性

- if 语句的“完备性”时，通常意味着通过分支覆盖了所有可能的条件，不遗漏任何可能的条件情况。
- 尽量理解问题领域，确保条件没有遗漏。尤其是在多个if-else的情况向。
- 从书写代码的方式来说，在最后提供一个无条件的else来作为安全网确保处理所有未明确处理的情况。

2.1.7 if语句的完备性

◆下面左面代码，在System.out.println的地方发生编译错误：

■the local variable y may not have been initialized

```
public class
E0211IfCompleteness {
    public static void
main(String[] args)
    {
        int x=5, y;
        if (x<0) {
            y=1;
        }
        if (x>=0) {
            y=2;
        }
        System.out.println(y
);
    }
}
```



```
public class
E0211IfCompleteness {
    public static void
main(String[] args)
    {//修改后编译错误消除
        int x=5, y;
        if (x<0) {
            y=1;
        } else {
            y=2;
        }
        System.out.println(y
);
    }
}
```

2.1.7 if语句的完备性

- 悬空else问题：if嵌套时，如果内层if没有else,就发生悬空看那个else问题。这时需要利用代码块包裹内部的if来达到目的。
- 下面第一个例子，人看起来是以为第一个if和else配对，其实编译器会把else和第二个if配对。

```
if ( x > 0 )
    if (y > 0)
        System.out.println("First case");
else
    System.out.println("Second case")
```

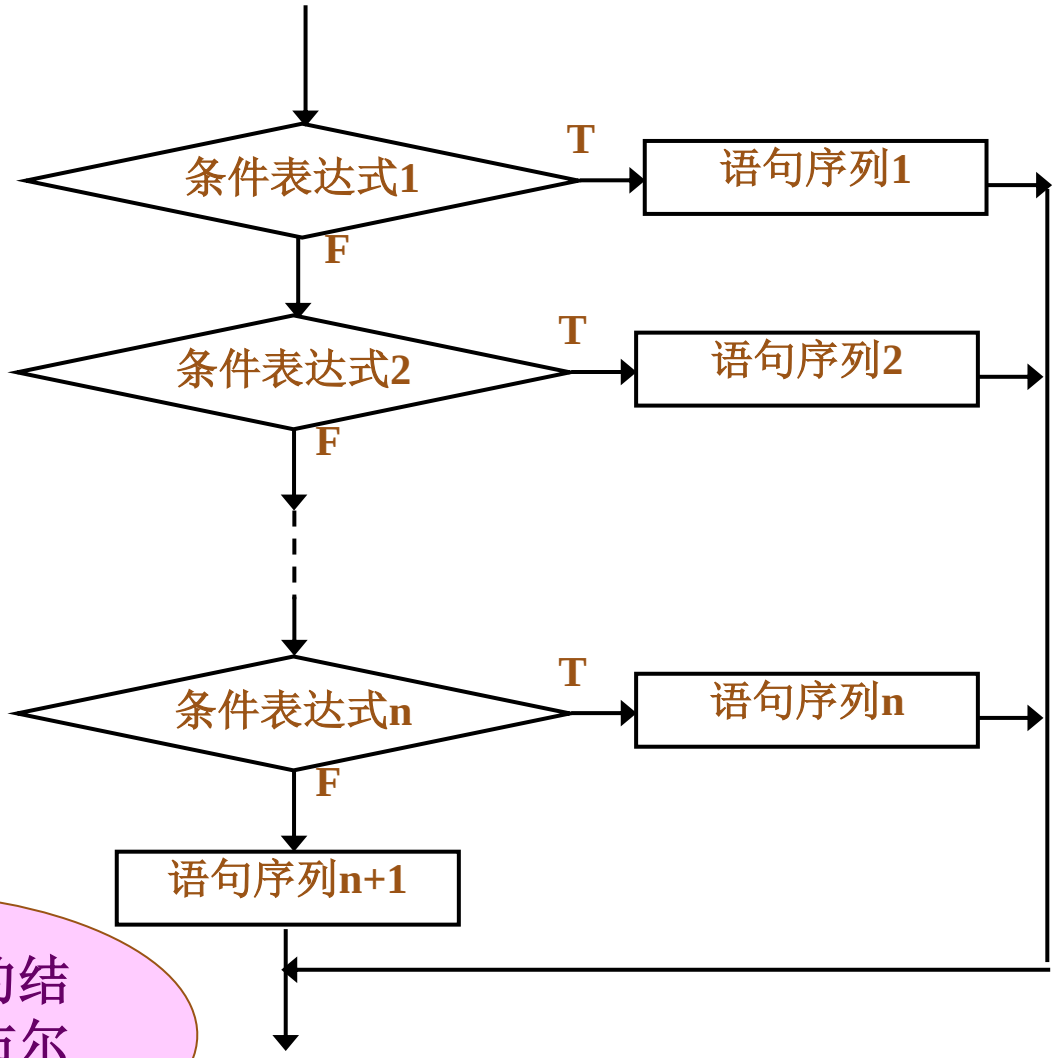


```
if ( x > 0 ) {
    if (y > 0)
      System.out.println("First case");
}
else
    System.out.println("Second case")
```

if多重条件选择结构（if-else嵌套）

```
if (条件表达式 1 )
{
语句序列 1
}
else if (条件表达式 2 )
{
语句序列2
}
. . . . . .
else if (条件表达式n )
{
语句序列n
}
else {
语句序列n+1
}
```

条件表达式的结果必须都是布尔型



多重条件选择结构

2.1.8 多路分支

◆已知一个学生的分数，给出其分数等级。90-100分为A级；80-89分为B级；70-79分为C级；60-69分为D级；0-59分为E级

```
public class E0212IfelseMultiDemo {
  public static void main(String[] args)  {
    int testScore=86;
    char grade;
    if(testScore>=90) {
      grade='A';
    } else if (testScore>=80) {
      grade='B';
    } else if (testScore>=70) {
      grade='C';
    } else if (testScore>=60) {
      grade='D';
    } else  {
      grade='E';
    }
    System.out.println("评定成绩为："+ grade);
  }}
```

程序输出结果为：
评定成绩为：B

2.1. 9 空语句

◆空语句有时候会出现在 Java 程序中，空语句是一个仅由分号组成的语句，程序运行时什么也不做。空语句的存在使得“}”之后的分号是合法的语句：

例一

```
if (x < 0) {
    x = -x;
};  //这个分号是一个独立语句，不是if语句的一部分
```

例二：if语句中使用空语句

```
if ( done )
    ; // 空语句。 条件成立什么也不做
else
    System.out.println( "Not Arrived.")
```

例三：多余的空语句导致结果问题。比如这个例子只是空循环10次，不会输出10次"Hello"

```
for (i = 0; i < 10; i++);
    System.out.println("Hello");
```

2.1.0 switch语句

switch多路选择

```
switch（表达式）
 {
   case 常量表达式1：
        语句序列1；
        break；
    case 常量表达式2：
        语句序列2；
        break;
    . . . . . .
   case 常量表达式n:
        语句序列n;
        break;
   default:
        语句序列n+1;
}
```

必须与各个case后面的常量表达式值的类型一致。类型：整型及其引用类，字符型及其引用类，Java 5开始可以使用枚举，Java 7开始可以是String

每个case一般都有break，若没有则多个case共享一个分支，叫做“滑落"(fall-trough)

表达式的值与任何一个case后的常量表达式的值不相同时执行。为了确保完备性，不省略default

2.1.0 switch语句

- 从Java 7开始，switch语句的expression可以是以下几种类型：
  - byte 和 Byte
  - short 和 Short
  - char 和 Character
  - int 和 Integer
  - Enum 类型
  - String 类型

◆根据季节进行描述季节景象。

```
public class E0213SwtichString {
    public static void main(String[] args) {
        String season = "秋天";
        switch (season) {
            case "春天":
                System.out.println("春暖花开.");break;
            case "夏天":
                System.out.println("夏日炎炎.");break;
            case "秋天":
                System.out.println("秋高气爽.");break;
            case "冬天":
                System.out.println("冬雪皑皑.");break;
            default:
                System.out.println("这啥季节啊不知道怎么描述
了。");
        }
    }}
```

程序输出结果：
秋高气爽.

2.1.0 switch语句*

- Java 14开始正式引入箭头语法和switch表达式。与原始的 switch 语法比较，这种新的 switch 格式与人类思维方式相近。
- 箭头语法：在case标签之后使用箭头符号（->），后跟执行的代码。如果匹配了case，则执行箭头右侧的代码，并自动结束case，这意味着之后的case不会被执行，这样就避免了传统switch中要求break语句来防止fall-through（穿透）的问题。每个箭头后面可以是一个表达式，一个代码块或者直接抛出一个异常。
- switch表达式的增强允许它不仅仅是执行一段代码，它可以计算并返回一个值。这意味着switch可以直接用于赋值操作，或者作为另一个表达式的一部分。
- switch表达式可以配合箭头语法使用。也可以使用 传统的case和冒号，但是这种情况必须配合yield来返回值。

2.1.0 switch语句*


```
int day=3;
String dayType = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid day";
};
System.out.println(dayType); //输出”Weekday"
```

```
String monthString = switch (month) {
    case 1:
        yield "January";
    case 2:
        yield "February";
    // ...更多的月份
    default:
        yield "Invalid month";
};
```

- 算法是程序执行任务所采取的步骤的想法。在描述算法时，可以用任何语言表达。当然，在完善算法的所有细节都后，才能用具体的语言转换为一个实际的程序。
- 假设有一个需要完成的计算任务，逐步细化的过程（自顶向下）如下：
  - 首先编写任务描述，并将该描述作为将要开发的算法的大纲。
  - 然后，细化和详细说明该描述，逐渐添加步骤和细节，直到可以直接翻译成编程语言的完整算法。
  - 随着逐步细化阶段的进行，可以用伪代码写出对算法的描述（即模仿编程语言结构的非正式指令，而没有实际程序代码的完整细节和语法）
  - 最后把伪代码转换为某一种程序设计语言。
    - 伪代码转换成任何具体的编程语言，比如Java 、 C/C++ 、 Python、JavaScript、Ruby、Go、Swift等，前提是该语言能够实现伪代码中描述的逻辑和功能。
    - 选择哪种编程语言通常取决于多种因素，如项目需求、性能考量、开发团队的熟悉度、可用的库和工具等。

2.2.1 伪代码和逐步细化

◆例如，程序执行的任务是：“根据用户指定的初始资金和利率，计算并显示未来五年每年的投资价值”。根据任务的需求，执行流程如下：

```
获取用户的初始资金和利率
计算 1 年后的投资价值
显示值
计算 2 年后的值
显示值
计算 3 年后的值
显示值
计算 4 年后的值
显示值
计算 5 年后的值
显示值
```



```
获取用户的初始资金和利率
while 投资期限处理(按照期限循环):
    计算下一年之后的值
    显示值
```

◆这个过程是正确的，但重复步骤太多。这时就应当考虑使用循环。使用循环的最大好处，是无论计算多少年，使用相同的循环就可以处理。

2.2.1 伪代码和逐步细化

◆按照上述的算法，就可以解决问题。但是对于计算机来说，需要更明确地解决按个步骤的细节：

- ■如何获取用户的初始资金和利率: 细化到具体步骤
- ■如何化计算下一年后的值： 把利率加到前一年的本金中
- ■如何判断是否已经处理了所有的年:在循环中计数

```
提示用户输入初始资金
读取用户的输入
提示用户输入利率
读取用户的输入
```

```
while 投资期限处理：
        计算利息
        将利息添加到本金中
        显示值
```

```
years = 0
while years < 5:
        years = years + 1
        计算利息
        将利息添加到本金中
        显示值
```

2.2.1 伪代码和逐步细化

◆假设利息是通过将利率乘以投资的当前金额来计算的，那么就可以计算利息。将其与获取用户输入的代码组合在一起，就有了完整的算法：

```
提示用户输入初始资金 value
读取用户的输入
提示用户输入利率 interest rate
读取用户的输入
years = 0
while years < 5:
        years = years + 1
        计算：  interest = value * interest rate
        把interest 加到value
        显示value
```

2.2.1 伪代码和逐步细化

◆然后把伪代码翻译为Java代码：

```
import java.util.Scanner;
public class E0214Interest5Years {
  public static void main(String[] args) {
    Scanner scanin = new Scanner(System.in);
    double principal;  // The value of the
investment.本金
    double rate;       // The annual interest
rate. 利率
    /* Get the initial investment and interest
rate from the user. */
    System.out.print("Enter the initial
investment: ");
    principal = scanin.nextDouble();
    System.out.println("Enter the annual
interest rate.");
    System.out.print("Enter a decimal, not a
percentage: ");
    rate = scanin.nextDouble();
    System.out.println("--------------------");
```

```
    /* Simulate the investment for 5
years. */
    int years;  // Counts the number
of years that have passed.
    years = 0;
    while (years < 5) {
      double interest;  // Interest
for this year.
      interest = principal * rate;
      principal = principal +
interest;     // Add it to principal.
      years = years + 1;    // Count
the current year.
      System.out.printf("The value of
the investment after %d years is
$%1.2f%n", years, principal);
    } // end of while loop
    scanin.close();
  } // end of main()
} // end of class Interest3
```

2.2.1 伪代码和逐步细化

- 在编写伪代码时，逐步细化的程度取决于几个因素，包括算法的复杂性、项目要求、预期的读者或团队成员的背景，以及伪代码将如何被使用。
- 伪代码的细化程度取决于使其既不至于过于抽象，以至于丢失重要信息，也不至于过于详细，以至于与具体编码无异。适当平衡才能达到最佳的沟通效果。
- 伪代码的核心目的是提高理解性和沟通效率，所以只要这一目的得到满足，伪代码所体现的水平细节就足够了。通常，伪代码应清楚地描述程序要完成什么事情，而不是具体如何实现所有细节。

2.2.1 伪代码和逐步细化

◆何时停止细化：

- 目标清晰：伪代码的目的是让读者理解算法的工作原理，所以一旦它足够清晰地表达了算法或程序逻辑，就可以停止细化。
- 步骤明确：你应根据读者的预期知识水平来判断。如果你的读者是熟悉编程的人，那么可以使用更抽象的表达方式。如果读者不那么熟悉编程概念，则可能需要更详尽的描述。
- 逻辑正确性：伪代码中的每一个逻辑步骤都应该是正确的。如果它可以逐步执行而不产生歧义，那么就细化到位了。
- 一致性：应用一致的逻辑和描述水平，避免在一个部分过于简略而在另一个部分异常详细。
- 复杂部分：如果伪代码中包含了特别复杂或不明显的算法部分，那么可能需要更详细的说明，甚至可能需要转化为步骤内的小块伪代码。
- 转换为程序的准备：实际编程中，伪代码应该足够详细，以便能够容易地转换为实际代码，但又不需要涉及特定编程语言的语法细节。

2.2.2 编码、编译、测试、调试

- 将算法转化为 Java 源代码的过程称为编码，它将算法设计翻译成用 Java 或其他语言编写的程序。
- 编码难免会引入一些错误。编译器会检查出所有的语法错误。
  - 但是编译器给出的语法错误又是并不精确。比如如果{}不配对，可能错误信息并不那么明确。
  - 一般地，如果有多个错误信息，尽量先修改第一个错误。因为其余的错误可能是第一个错误引起的。
  - 在修改错误之前，要尽量理解这个错误。尤其是要结合自己的
- 输入代码时候遵循一些基本的方针，会避免很多错误。
  - 比如输入{时候顺便输入},然后在中间插入内容（IDE已经具备这个功能)
  - 遵循统一的命名模式
  - 良好的代码缩进和对齐

2.2.2 编码、编译、测试、调试

◆编译通过后，就需要进行测试，以确保工作正常。

- 不是简单地测试两组输入，而是要确保对所有合理的输入都得到正确的结果。
- 理想情况下，对于不合理的输入，应当优雅地处理这种情况，例如显示一条友好的错误消息，引导用户重新输入正确的数据，而不是程序崩溃。
- 需要测试各种输入组合，尽可能设计出覆盖程序所有功能的输入组合进行测试。
- 测试的重点是找出bug, 也就是程序处理结果错误。
  - 编写程序无法完全避免引入bug.
- 为了找到bug的具体位置，就需要调试(debug)
  - 首先阅读代码，设想代码执行的结果。然后一步一步排查，看看代码执行结果是否符合预期。
  - 可以设置断点，查看变量，查看内存，查看调用栈等等。
  - 还可以临时插入一些System.out.println() ，并执行程序，观察输出的变量的值是否符合预期。

2.2.3 算法设计例题: (1)3N+1问题

- 3N+1问题，又称为Collatz猜想、奇偶归一猜想、冰雹猜想，是数学上的一个未解决问题，它是关于整数序列行为的猜想。这个问题可以被描述为对于任意正整数N，按照以下步骤进行操作：
  - 如果N是偶数，将其除以2（N = N / 2）。
  - 如果N是奇数，将其乘以3并加1（N = 3 * N + 1）。
  - 重复步骤1和2，生成一个序列。
- Collatz猜想声称无论选择哪个大于零的整数作为序列的起始数字，该序列最终都会达到1。一旦序列包含了数字1，它就会进入循环：4, 2, 1。
- 举个例子，如果从6开始，生成的序列将会是：6, 3, 10, 5, 16, 8, 4, 2, 1。

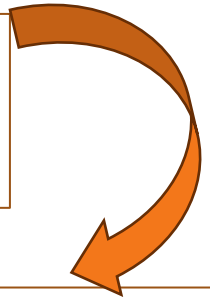
2.2.3 算法设计例题: (1)3N+1问题

◆编写一个程序，从用户那里读取一个正整数，并从该正整数开始打印出3N+1序列和序列中的项数。(序列到1为止)

◆算法概要如下面第一张图

◆这个算法主要内容在第二步。利用循环对第二步进一步细化，写出伪代码。

从用户那里获取正整数N
计算、输出序列中的每个数，并对其计数
输出计数结果



```
从用户那里获取正整数N;
while N is not 1:
    计算：N = next term;
    输出：  N;
    计数这一项;
输出计数结果;
```

2.2.3 算法设计例题: (1)3N+1问题

◆"计算：N = next term;"这一步需要进一步细化。为此，引入if语句细化如下:

```
从用户那里获取正整数N;
while N is not 1:
    计算：N = next term;
    输出：  N;
    计数这一项;
输出计数结果;
```



```
从用户那里获取正整数N;
while N is not 1:
    if N is even(或者写：如果N是偶数）：
        计算: N = N/2;
    else
        计算：  Compute N = 3 * N + 1
    输出：  N;
    计数这一项;
输出计数结果;
```

◆这时，伪代码已经基本达到编写程序的要求。但是对于有些编程不熟练的读者来说，还需要进一步细化。

- 计数怎么实现: 设置一个变量，在循环开始前赋值为0.每次需要计数时对该变量加一。
- 确保用户输入的是一个正整数。为此，引入一个循环来取得用户输入。

2.2.3 算法设计例题: (1)3N+1问题

◆最终细化的算法如下:

```
Ask user to input a positive number;
Let N be the user’ s response;
while N is not positive:
    Print an error message;
    Read another value for N;
Let counter = 0;
while N is not 1:
    if N is even:
        Compute N = N/2;
    else
        Compute N = 3 * N + 1;
    Output N;
    Add 1 to counter;
Output the counter;
```

2.2.3 算法设计例题: (1)3N+1问题

◆算法对应的Java代码:

```
import java.util.Scanner;
public class E0215ThreadN1 {
  public static void main(String[] args)
{
    Scanner scanin = new
Scanner(System.in);
    int N;        // for computing terms
in the sequence
    int counter; // for counting the
terms
    System.out.print("请输入序列起点: ");
    N = scanin.nextInt();
    while (N <= 0) {
        System.out.print("序列起点必须是
正整数，请重新输入: ");
        N =  scanin.nextInt();
    }
    // At this point, we know that
N &gt; 0
    counter = 0;
    while (N != 1) {
        if (N % 2 == 0)
            N = N / 2;
        else
            N = 3 * N + 1;
        System.out.println(N);
        counter = counter + 1;
    }
    System.out.println("序列中有
"+counter+"项。");

    scanin.close();
  }  // end of main()
}  // end of class
```

2.2.3 算法设计例题：（2）while 语句的综合使用

- 求用户输入的一组正整数的平均值。平均值是正整数的和除以正整数的个数。伪代码。
- 用户一次输入一个整数，程序记录输入的整数个数，并且计算所有去读的数字的和。程序的伪代码算法如下面图所示：

```
Let sum = 0 // 用户输入的整数之和.
Let count = 0 // 用户输入的整数个数.
while there are more integers to process:
    Read an integer
    Add it to the sum
    Count it
Divide sum by count to get the average
Print out the averag
```

2.2.3 算法设计例题：（2）while 语句的综合使用

◆这个算法中，如何判断 while 循环结束是问题的关键。

- 一个解决方案是在输入所有数据后，最后一次用户输入 0，while 循环读入数据 0 时，结束循环。所以 while 循环中的条件变为“当输入整数不为零时”。
- 因为用的是while循环，需要启动循环。也就是在进入循环之前按先输入一个正整数。

```
Let sum = 0 // 用户输入的整数之和.
Let count = 0 // 用户输入的整数个数.
while there are more integers to process:
    Read an integer
    Add it to the sum
    Count it
Divide sum by count to get the average
Print out the average
```



```
Let sum = 0 // 用户输入的整数之和.
Let count = 0 // 用户输入的整数个数.
Read an integer
while the integer is not zero:
    Add the integer to the sum
    Count it
    Read an integer
Divide sum by count to get the average
Print out the average
```

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

◆转换后的Java程序如下

```
import java.util.Scanner;
public class E0216ComputeAverage {
  public static void main(String[] args) {
    int inputNumber; //用户输入的一个整数.
    int sum;         // 整数的和.
    int count;       // 整数的个数.
    double average;  //平均值.
    Scanner scanin = new Scanner(System.in);
    /* 初始化sum和count. */
    sum = 0;    count = 0;

    /* 读取用户输入. */
    System.out.print("Enter your first
positive integer: ");
    inputNumber = scanin.nextInt();
    while (inputNumber != 0) {
        sum += inputNumber;    // 累加和.
        count++;               //计数.
        System.out.print("Enter your next
positive integer, or 0 to end: ");
        inputNumber = scanin.nextInt();
    }

    /* Display the result. */
    if (count == 0) {
        System.out.println("You didn't
enter any data!");
    }
    else {
        average = ((double)sum) / count;
        System.out.println("\nYou entered
" + count + " positive integers.");
        System.out.printf("Their average
is %1.3f.%n", average);
    } } // end main()
} // end class ComputeAverage
```

2.2.3 算法设计例题：（3）for 循环的综合使用

◆编写一个程序，从用户那里输入一个正整数 N，并计算 N 可以被多少个不同的数整除

◆一种解决方案 N 是：对于 N 的可能除数集合 D={1, 2, . . . , N} 逐一检查，如果 D 中的元素 x 使得 N % x 为零，那么 x 就是 N 的一个除数。伪代码如下第一张图。for循环对应的Java代码在第二张图中：

```
获取用户输入的整数 N
计数器divisorCount=0
对1..N中的每一个数testDivisor:
    如果testDivisor可以整除 N:
        计数器divisorCount加一
输出divisorCount
```

```
for (testDivisor = 1; testDivisor <= N; testDivisor++) {
    if ( N % testDivisor == 0 ){
    }     divisorCount++;
}
```

2.2.3 算法设计例题：（3）for 循环的综合使用

◆对于很大的输入N, 计算会花一些时间，因此对算法改进一下，在运算中提示进度：再引入一个计数器，每测试过10000000个数，显示一个".",然后这个新引入的计数器归零从头计数。

```
Get a positive integer, N, from the user
Let divisorCount = 0 // Number of divisors found.
Let numberTested = 0 // Number of possible divisors tested
// since the last period was output.
for each number, testDivisor, in the range from 1 to N:
    if testDivisor is a divisor of N:
        Count it by adding 1 to divisorCount
    Add 1 to numberTested
    if numberTested is 10000000:
        print out a ‘ .’
        Reset numberTested to 0
Output the count
```

◆转换后的Java程序在下页

2.2.3 算法设计例题：（3）for 循环的综合使用

```
import java.util.Scanner;
public class E0217CountDivisors {
  public static void main(String[] args)
{
    int N;  // 用户输入的正整数
    int testDivisor;  // 1..N之间的一个数
    int divisorCount;  // 能整除的计数器.
    int numberTested;  // 计数测试数
    Scanner scanin =
new  Scanner(System.in);
    /* 输入一个正整数. */
    while (true) {
        System.out.print("Enter a
positive integer: ");
        N = scanin.nextInt();
        if (N > 0)
            break;
        System.out.println("That number
is not positive.  Please try again.");
    }
```

```
    /* 计算被除数个数. */
    divisorCount =
0;    numberTested = 0;
    for (testDivisor = 1;
testDivisor <= N; testDivisor++) {
        if ( N % testDivisor ==
0 )  divisorCount++;
        numberTested++;
        if (numberTested ==
10000000) {
            System.out.print('.');
            numberTested = 0;
        }
    }

    System.out.println("\nThe
number of divisors of " + N
            + " is " +
divisorCount);
  } // end main()
} // end class CountDivisors
```

2.2.3 算法设计例题：（3）for 循环的综合使用

```
import java.util.Scanner;
public class E0217CountDivisors {
  public static void main(String[] args)
{
    int N;  // 用户输入的正整数
    int testDivisor;  // 1..N之间的一个数
    int divisorCount;  // 能整除的计数器.
    int numberTested;  // 计数测试数
    Scanner scanin =
new  Scanner(System.in);
    /* 输入一个正整数. */
    while (true) {
        System.out.print("Enter a
positive integer: ");
        N = scanin.nextInt();
        if (N > 0)
            break;
        System.out.println("That number
is not positive.  Please try again.");
    }
```

```
    /* 计算被除数个数. */
    divisorCount =
0;    numberTested = 0;
    for (testDivisor = 1;
testDivisor <= N; testDivisor++) {
        if ( N % testDivisor ==
0 )  divisorCount++;
        numberTested++;
        if (numberTested ==
10000000) {
            System.out.print('.');
            numberTested = 0;
        }
    }

    System.out.println("\nThe
number of divisors of " + N
            + " is " +
divisorCount);
  } // end main()
} // end class CountDivisors
```

2.2.3 算法设计例题：（3）for 循环的综合使用

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

◆for的循环嵌套

- ■例如，输出一个12x12的乘法表，伪代码如下图.
- ■对其中的“输出12个数进一步细化”，得到右面的结果。

```
for each rowNumber = 1, 2, 3, ..., 12:
    输出 12 个数
    输出回车换行符
```



```
for each rowNumber = 1, 2, 3, ..., 12:
    for N = 1, 2, 3, ..., 12:
        Print N * rowNumber
    Output a carriage return
```

1	2	3	4	5	6	7	8	9	10	11	12
2	4	6	8	10	12	14	16	18	20	22	24
3	6	9	12	15	18	21	24	27	30	33	36
4	8	12	16	20	24	28	32	36	40	44	48
5	10	15	20	25	30	35	40	45	50	55	60
6	12	18	24	30	36	42	48	54	60	66	72
7	14	21	28	35	42	49	56	63	70	77	84
8	16	24	32	40	48	56	64	72	80	88	96
9	18	27	36	45	54	63	72	81	90	99	108
10	20	30	40	50	60	70	80	90	100	110	120
11	22	33	44	55	66	77	88	99	110	121	132
12	24	36	48	60	72	84	96	108	120	132	144

2.2.3 算法设计例题：（3）for 循环的综合使用

◆要求矩阵的输出整齐，规定每个输出数据占用四个位置，可以使用带有格式说明符%4d。假设已经将 rowNumber 和 N 声明为 int 类型的变量，那么该算法在 Java 中可以表示为

```
for ( rowNumber = 1; rowNumber <= 12; rowNumber++ ) {
    for ( N = 1; N <= 12; N++ ) {
        // 每列按照 4 个字符格式输出
        System.out.printf( "%4d", N * rowNumber ); //不输出回车换行符
    }
    System.out.println(); // 在一行的末尾输出一个回车换行符.
}
```

2.2.3 算法设计例题：（3）for 循环的综合使用

◆例：查找给定字符串中包含的字母。比如说，字符串“Hello World”中出现过的字母是 D、E、H、L、O、R 和 W。更具体地说，编写一个程序，根据用户输入的字符串，程序列出字符串中包含的所有字母，并计算不同字母的数量.

```
提示用户输入一个字符串
读取用户的输入到字符串变量，str
Let count = 0 (count 用来存放字不同字母出现过的数量)
对每一个字母letter循环处理:
    如果该字母letter出现在 str:
        输出字母letter
        count +1
输出计数器
```

◆对伪代码进一步细化:

■为了避免同时需要检查大写和小写字母，假设对于字符的大小写不敏感。可以通过在处理 str 之前将它转换为大写来避免两次处理。这样的话，只需要检查大写字母就可以

2.2.3 算法设计例题：（3）for 循环的综合使用

- “循环处理字符表中的每一个字符”实现为“for (letter='A'; letter<='Z';letter++)”。
- “如果该字母letter出现在 str”代表检查给定的字符 letter 是否出现在 str 中。一种方式是依次查看 str 字符串中的每个字符，并检查该字符是否与当前检查的字符letter相等。可以通过函数调用 str.charAt(i)获取str 的第 i 个字符，其中 i 的范围从 0 到 str.length()-1。

```
提示用户输入一个字符串
读取用户的输入到字符串变量，str
将 str 转换为大写字符
Let count = 0 (count 用来存放字不同符出现过的数量)
for letter = ’A’, ’B’, ..., ’Z’:
    for i = 0, 1, ..., str.length()-1:
        if letter == str.charAt(i):
            输出字符
            count+1
            break;
输出计数器 count
```

2.2.3 算法设计例题：（3）for 循环的综合使用

■伪代码转换为Java代码：

```
import java.util.Scanner;
public class E0218ListLetters {
  public static void main(String[]
args) {
      String str;  // Line of text
entered by the user.
      int count;   // Number of
different letters found in str.
      char letter; // A letter of the
alphabet.
      Scanner scanin= new
Scanner(System.in);
      System.out.println("Please type
in a line of text.");
      str = scanin.nextLine();
      str = str.toUpperCase();
      count = 0;
      System.out.println("Your input
contains the following letters:\n");
      System.out.print("   ");
      for ( letter = 'A'; letter <=
'Z'; letter++ ) {
        for ( int i = 0; i <
str.length(); i++ ) {
          if ( letter ==
str.charAt(i) ) {
            System.out.print(letter+
" ");
            count++;
            break;
          }
        }
      }
      System.out.println("\n\nThere
were " + count + " different
letters.");
      scanin.close();
   } // end main()
} // end class ListLetters
```

2.2.3 算法设计例题：（4）if 语句综合应用

◆假设 x、y 和 z 是 int 类型的变量，并且每个变量都已被赋值。程序的任务是按递增顺序输出三个变量的值。例如，如果值为 42、17 和 20，则输出的顺序应为 17、20、42。

◆方法一:确定每个值在列表中的位置。如果 x 小于 y 和 z，则 x 排在列表中的第一位。如果它大于 y 和 z，则排在列表的最后。否则，它会出现在 y 和 z 的中间。我们可以用 3-分支 if 语句来表达这一点，但程序还需要判断 y 和 z 的输出顺序。伪代码：

```
if (x < y && x < z) {
    输出 x, 紧跟着是 y 和 z
}
else if (x > y && x > z) {
    输出 y 和 z,紧跟着是 x
}
else {
    在 y 和 z 之间输出 x
}
```

2.2.3 算法设计例题：（4）if 语句综合应用

◆为了确定 y 和 z 的输出顺序，需要另一个 if 语句，以上代码修改为：

```
if (x < y && x < z) { // x 是第一个数
    if (y < z)
        System.out.println( x + " " + y + " " + z );
    else
        System.out.println( x + " " + z + " " + y );
}
else if (x > y && x > z) { // x 是最后一个数
    if (y < z)
        System.out.println( y + " " + z + " " + x );
    else
        System.out.println( z + " " + y + " " + x );
    }
else { // x 位于中间
    if (y < z)
        System.out.println( y + " " + x + " " + z);
    else
        System.out.println( z + " " + x + " " + y);
}
```

2.2.3 算法设计例题：（4）if 语句综合应用

◆方法二：首先要确定“x 和 y 应该按什么顺序输出？一旦知道了，你只需要决定在哪里放置 z。这种思路导致了不同的 Java 代码：

```
if ( x < y ) { // x 在 y 之前
    if ( z < x ) // z 在最前面
        System.out.println( z + " " + x + " " + y);
    else if ( z > y ) // z 在最后
        System.out.println( x + " " + y + " " + z);
    else // z 在中间
        System.out.println( x + " " + z + " " + y);
}
else { // y 在 x 前面
    if ( z < y ) // z 在最前面
        System.out.println( z + " " + y + " " + x);
    else if ( z > x ) // z 在最后
        System.out.println( y + " " + x + " " + z);
    else // z 在中间
        System.out.println( y + " " + z + " " + x);
}
```

2.2.3 算法设计例题：（4）if 语句综合应用

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

◆编写一个程序，将重量从一种单位转换为另一种，例如克、斤、千克到吨的转换。用户将以这些单位之一输入测量值，例如“1700jin”或“0.85 ton”。输出将根据四个测量单位中的每一个显示长度。换算过程概要如下：

读取用户的输入测量值和测量单位
重量换算为grams、jin、 kilogram和ton四种单位
显示 4 种换算结果

◆使用 input.nextDouble()读取数值测量值和 input.nextLine()读取重量单位。

◆由于克的数量可以很容易地转换成斤、千克和吨，首先将用户的输入转换为克，然后再转换到不同的单位换。

◆在转换为克之前，我们必须检查输入值和用户指定的单位：

2.2.3 算法设计例题：（4）if 语句综合应用

◆细化后的伪代码：

```
Let measurement = input.nextDouble()
Let units = input.nextLine()
if units is grams
    Let grams = measurement
else if units is jin
    Let grams = measurement * 500 // 每斤等于 500 克
else if is kilogram
    Let grams = measurement * 1000 // 每千克等于 1000 克
else if units is ton
    Let grams = measurement * 2 *1000 // 每吨等于2000 克
else
    不存在的测量单位。
    显示错误信息并返回
Let jin = grams / 500.0
Let kilogram = grams / 1000.0
Let ton = grams / (2.0 * 1000.0)
输出结果
```

2.2.3 算法设计例题：（4）if 语句综合应用

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- 由于单位是一个字符串，可以使用 units.equals("grams")来检查指定的度量单位是否为“克”。
- 如果允许用户将“grams”缩写为“g”。那么 if 语句就需要使用逻辑运算符中的条件或“||”来组成复合条件表达式，即 if (units.equals("grams") || units.equals("g"))。
- 如果还允许输入大写字母，即“grams”或“GRAMS”等价，可以通过将单位字符串转换为小写或用函数 units.equalsIgnoreCase 代替 units.equals 来实现。
- 为了让用户重复输入测量值并查看每个测量值的转换结果，所以使用了 while 循环。当用户输入 0 时循环结束。
- 用户的输入可能非常大或也可能非常小，所以使用带有“g”格式符的输出。当输入数据非常大或非常小，“g”格式将输出一个指数形式的实数，否则的话，以通常的十进制形式打印。
- 在格式规范%14.5g 中，5 是有效数字的位数，即要输出的数字的位数，因此无论数字大小如何，将始终在输出中获得相同数量的有效数字。比如对于数字0.000000000745482，如果使用了“f”格式说明符，例如%14.5f，则输出将为小数点后 5 位的十进制形式，其输出为 0.00000，根本没有有效数字！相反，如果使用“g”格式说明符，输出将为 7.4549e-10。

2.2.3 算法设计例题：（4）if 语句综合应用

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

```
import java.util.Scanner;
public class E0219UnitConverter {
  public static void main(String[] args) {
    double measurement; // 用户输入的重量数字
    String units; // 用户指定的输入单位
    double grams, jin, kilogram, ton; //每种可能的度量单位的表示
    System.out.println("Enter measurements in grams, jin, kilogram,
or ton.");
    System.out.println("I will convert your input into the other
units");
    System.out.println("of measure(e.g: 500 g).");
    System.out.println();
    Scanner scanin=new Scanner(System.in);
    while (true) {
        /* 取得用户输入并把单位转换成小写字母 */
        System.out.print("Enter your measurement, or 0 to end: ");
        measurement = scanin.nextDouble();
        if (measurement == 0)   break; // 结束 while 循环
        units = scanin.nextLine();
        units = units.trim().toLowerCase(); // 删除空白，将单位转换为
小写
```

2.2.3 算法设计例题：（4）if 语句综合应用

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

```
        /* 将输入测量值转换为克 */
        if (units.equals("g") || units.equals("grams")) {
            grams = measurement;
        }else if (units.equals("JIN") || units.equals("jin")) {
            grams = measurement * 500;
        } else if (units.equals("kilogram") || units.equals("KG")) {
            grams = measurement * 1000;
        } else if (units.equals("TON") || units.equals("ton")) {
            grams = measurement * 2 * 1000;
        }         else {
            System.out.println("Sorry, but I don’t understand \""+
units + "\".");
            continue; // 返回到 while 循环的开始
        }
        /* 将克换算为斤、千克和吨 */
        jin = grams / 500;
        kilogram = grams / 1000;
        ton = grams / (2*1000);
```

2.2.3 算法设计例题：（4）if 语句综合应用

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

```
            /* 以各个单位输出测量值 */
            System.out.println("\nThat's equivalent to:");
            System.out.printf("%14.5g grams%n", grams);
            System.out.printf("%14.5g jin%n", jin);
            System.out.printf("%14.5g kilogram%n", kilogram);
            System.out.printf("%14.5g ton%n%n", ton);
            System.out.println();
        } // end while
        System.out.println();
        System.out.println("OK! Bye for now.");

        scanin.close();
    } // end main()
} // end class LengthConverter
```

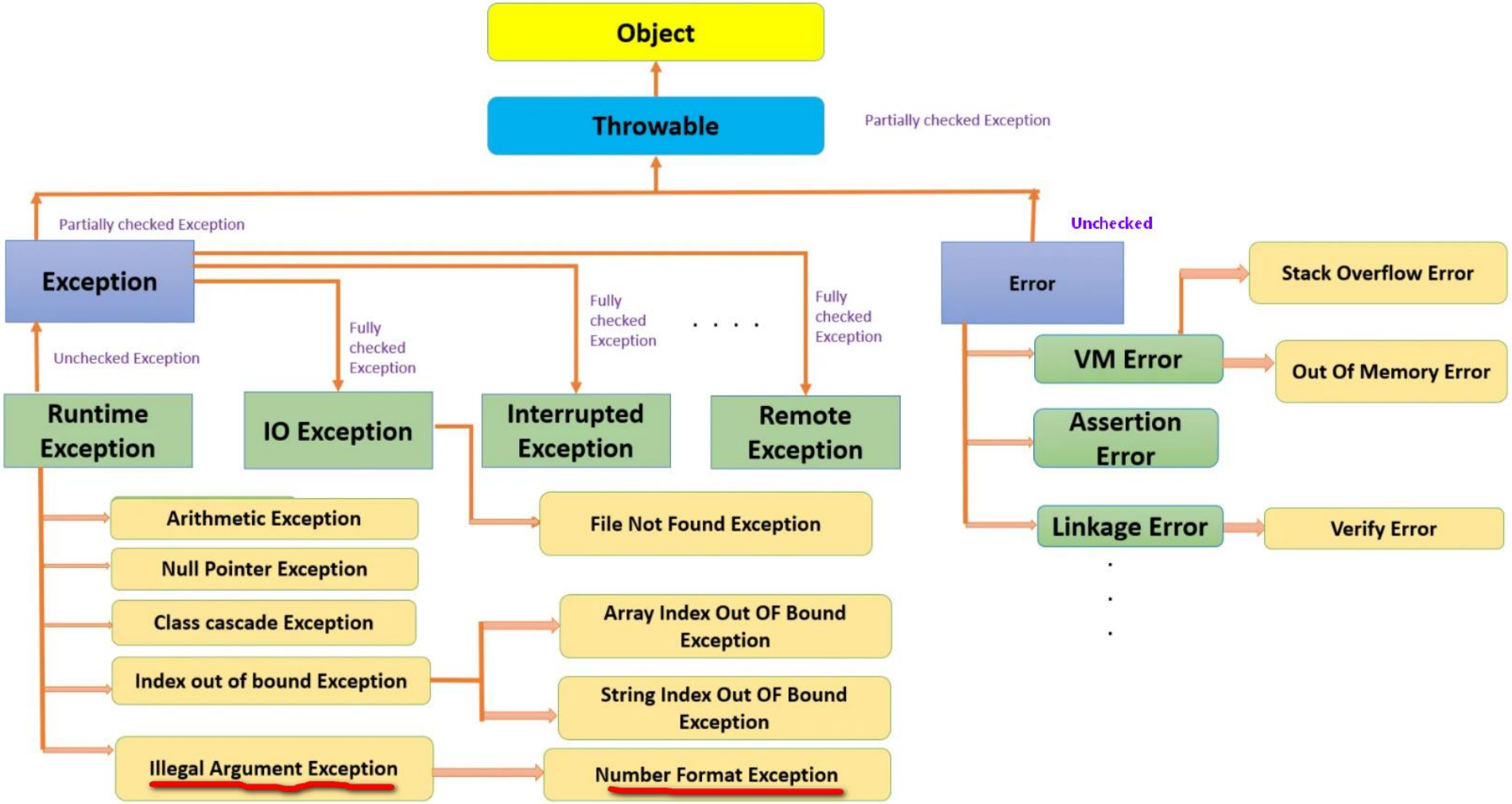
2.3 Java 语言中的异常处理简介

- 在 Java 语言中，除了具有正常控制流的控制结构之外，还能够处理“异常”控制流的控制结构。当程序执行期间发生错误时，Java 语言可以“捕获”此类错误并执行自己指定的响应，而不仅仅是让程序崩溃。
- 捕获并处理异常：try-catch
- 错误处理代码与业务代码分离。

```c
//C语言
FILE* fp=fopen(fn, "rt");
if (NULL==fp) {
    perror("文件打开失败。")
    goto err_exit;
}
```

```java
try (BufferedReader reader = new BufferedReader(new FileReader(fn)))
{
    //读入文件中数据进行处理
} catch (IOException e) {
    System.err.println("文件打开失败: " + fn);
    System.err.println(e.toString());
    e.printStackTrace();
}  //Java
```

◆Java异常的层次结构

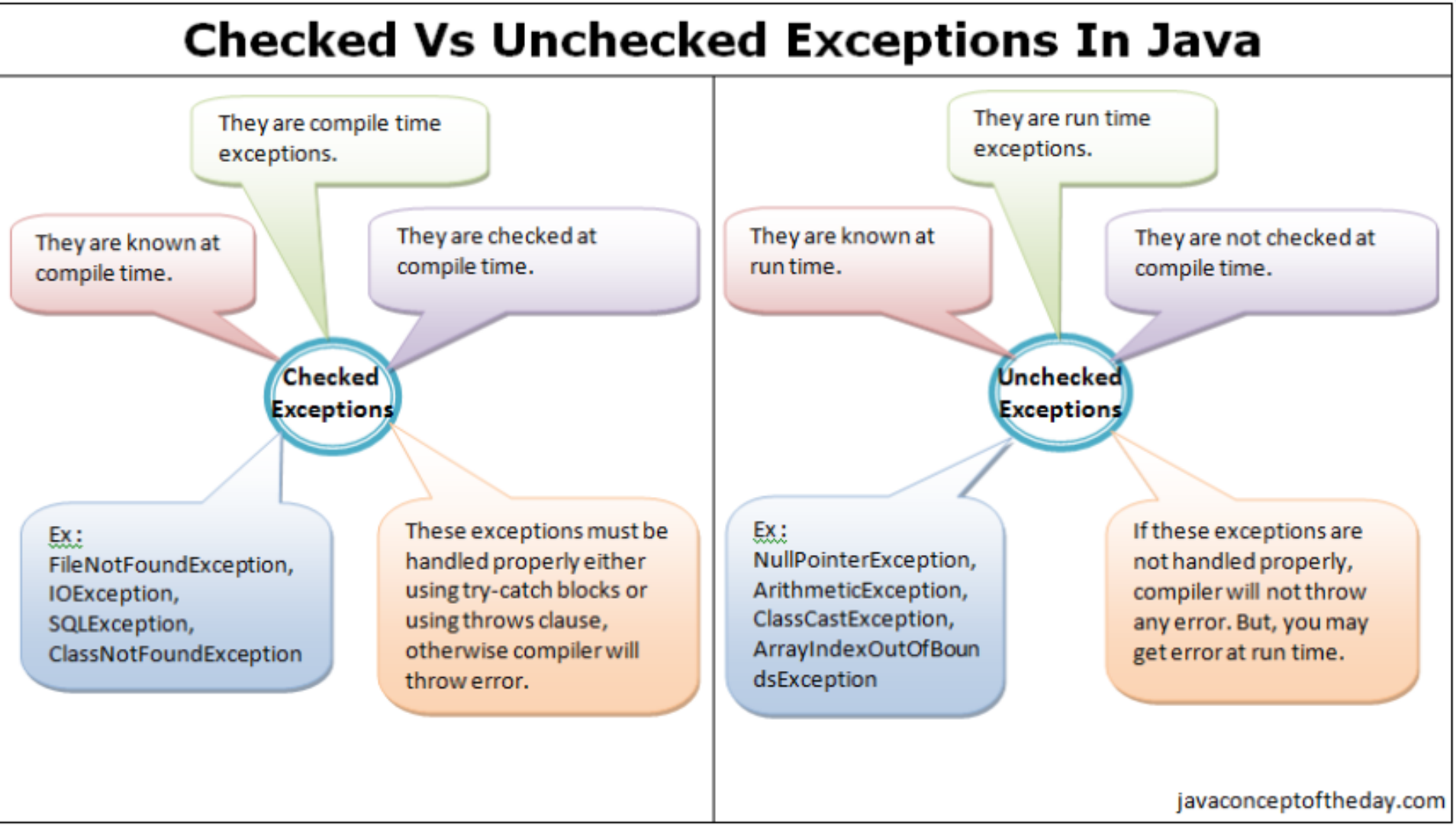


2.3.1 Exceptions

- ◆Java异常的分类: 检查型异常和非检查型异常
- ◆检查型异常（Checked Exceptions）
  - ■Checked的意思是编译器会检查代码中是否对这类异常进行了处理。
  - ■开发者必须通过try-catch块来处理这些异常，或者在方法签名中使用throws子句声明方法可能抛出这些异常，否则编译器会报错。
  - ■检查型异常通常是外部错误，让程序员有机会处理这些异常，以使程序能够恢复正常执行或者优雅地失败。
  - ■它们继承自 Exception 类，但不包括RuntimeException 及其子类。
- ◆检查型异常的例子包括：
  - ■IOException（如文件未找到时抛出的 FileNotFoundException）
  - ■SQLException（数据库访问错误）
  - ■ClassNotFoundException（类加载错误）

2.3.1 Exceptions

- ◆非检查型异常（Unchecked Exceptions）
  - ■Unchecked的意思是代码中是否对这些异常进行了处理，编译器不检查。
  - ■非检查型异常包括运行时异常（RuntimeException）和错误（Error）。
    - □RuntimeException及其子类：由程序逻辑错误导致的异常，如空指针访问、数组越界等。
    - □Error类及其子类: 代表与Java运行时系统本身相关的问题，由JVM生成和抛出的严重问题。程序不太可能从这些错误中恢复，因此通常不需要程序去处理这类情况。
- ◆运行时异常（RuntimeException）的例子包括：
  - ■NullPointerException
  - ■IndexOutOfBoundsException
- ◆错误的例子包括：
  - ■OutOfMemoryError
  - ■StackOverflowError
- ◆非检查型异常通常是编程错误或运行时无法预料的异常，不需要显式地在每个方法的签名中声明。

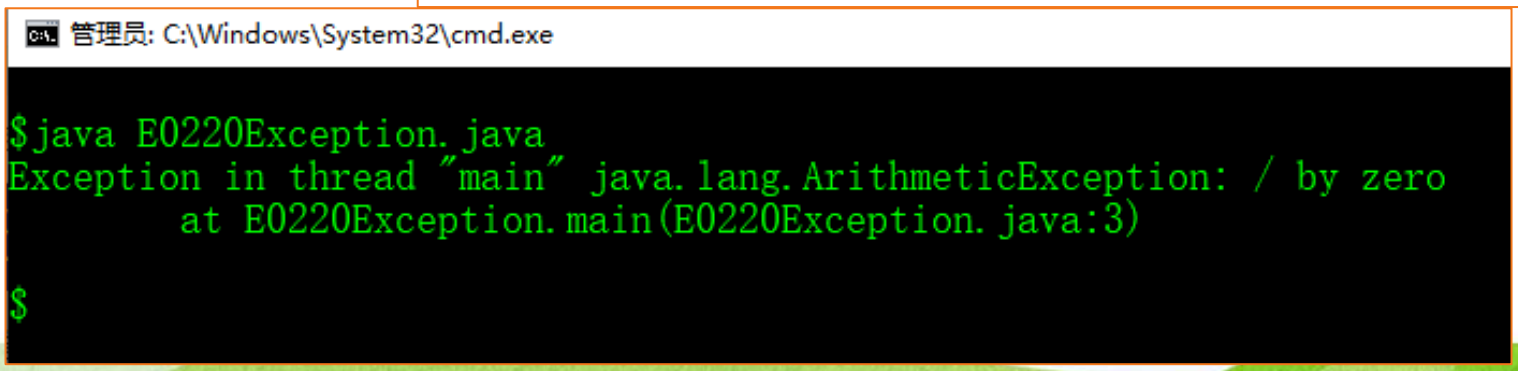

Checked Vs Unchecked Exceptions In Java
They are compile time exceptions.
They are known at compile time.
They are checked at compile time.
Checked Exceptions
Ex : FileNotFoundException, IOException, SQLException, ClassNotFoundException
These exceptions must be handled properly either using try-catch blocks or using throws clause, otherwise compiler will throw error.
They are run time exceptions.
They are known at run time.
They are not checked at compile time.
Unchecked Exceptions
Ex : NullPointerException, ArithmeticException, ClassCastException, ArrayIndexOutOfBoundsException
If these exceptions are not handled properly, compiler will not throw any error. But, you may get error at run time.
javaconceptoftheday.com

- 当使用函数 Integer.parseInt 和 Double.parseDouble 尝试将字符串转换为数字时，可能会发生 NumberFormatException。
  - 考虑函数调用 Integer.parseInt(str)，其中 str 是 String 类型的变量。如果 str 的值为字符串“42”，则函数调用将正确地将字符串转换为 int 42。但是，如果 str 的值为“fred”，则函数调用将失败，因为“fred”不是 int 值的合法字符串表示形式。在这种情况下，会发生 NumberFormatException 类型的异常。但是，如果不采取任何措施来处理异常，程序就会崩溃。
- 当非法值作为参数传递给函数时，可能会发生 IllegalArgumentException。
  - 例如，如果函数要求参数大于或等于 0，则在将负数传递给子例程时可能会发生 IllegalArgumentException。
  - 当然，如何响应非法值取决于函数的定义，所以不能简单地说每个非法参数值都会导致IllegalArgumentException。

2.3.2 try-catch

- 异常发生称为异常被“抛出”。例如，在 str 的值非法时，Integer.parseInt(str)抛出NumberFormatException 类型的异常。当抛出异常时，可以“捕获”异常来防止程序崩溃。
- 如果异常没有被当前方法捕获，会沿着调用栈层层向上层方法传递。如果最上层main也没有捕获，则由JVM捕获。
- JVM默认异常处理：JVM会进行默认的异常处理，这通常意味着异常信息会被打印到标准错误流（通常是控制台），包括异常的类型、错误信息以及堆栈跟踪。然后，JVM会终止程序执行，因为它已经遇到了一个未处理的异常。

```
public class E0220Exception {
    public static void main(String[] args) {
        System.out.println(1/0);
    }
}
```



异常使用5个关键字：try、catch、finally、throws、throw来处理异常。

1）使用try-catch-finally语句捕获异常；

2）通过throws子句声明异常；

3）使用throw抛出异常对象。



1、使用try-catch-finally语句捕获异常

用try-catch-finally语句来捕获一个或多个异常，基本格式为：

```
try{
    … //执行代码块
 }catch(ExceptionType1 e1){
    … //对异常类型1的处理
 }catch(ExceptionType2 e2){
    … //对异常类型2的处理
 }
 …
 catch(ExceptionTypeN eN){
    … //对异常类型N的处理
 }
 finally{
    … //异常发生，方法返回之前，总是要执行的代码（收尾工作）
 }
```



说明：

1）try语句用大括号{}指定了一段代码，该段代码可能会抛弃一个或多个异常。

2）try所限定的代码中，当抛弃一个异常时，其后的代码不会被执行。

3）catch所处理的异常类与生成的异常对象的类型完全一致或者是它的子类。比如catch (水果 ex), 那么苹果也会被catch.

4）无论try所指定的程序块中是否抛出异常，也无论catch语句的异常类型是否与所抛弃的异常的类型一致，finally所指定的代码都要被执行，它提供了统一的出口。

5）通常在finally语句中可以进行资源的清除工作。如关闭打开的文件等。

6) 异常对象可以调用如下方法得到或输出有关异常的信息：

- public String getMessage();
- public void printStackTrace();
- public String toString(); //这个比getMessage()详细



2.3.2 try-catch

◆例如，假设 str 是 String 类型的变量，其值可能表示也可能不表示合法的实数。那么下面代码段：

```
double x;
try {
    x = Double.parseDouble(str);
    System.out.println( "The number
is " + x );
}
catch ( NumberFormatException e ) {
    System.out.println( "Not a
legal number." );
    x = Double.NaN;
}
```

◆如果调用 Double.parseDouble(str)引发错误，则跳过 try 部分中的输出语句，并执行 catch部分中的语句。

◆在这个例子中，当异常发生时，程序将 x 设置为值 Double.NaN（Double.NaN是 double 类型的“not-a-number”值）。

2、通过throws子句声明异常

在有些情况下，一个方法并不需要处理它所生成的异常，而是向上传递，由调用它的方法来处理这些异常，这时就要用到throws子句。它包含在方法的声明中，其格式如下：

returnType methodName([paramlist]) throws exceptionList

exceptionList中可以声明多个异常，用逗号隔开。

通过throws子句声明异常说明：如果异常没有在当前的try...catch中处理，则它必须将异常抛弃给上一级调用它的方法。如果异常被回溯到main方法仍旧没有处理，则程序将出错，终止运行。注意：对非运行时例外，必须捕获或声明。



3、使用throw抛出异常对象

在捕获一个异常前，必须有一段Java代码生成一个异常对象并把它抛出。抛出异常的代码可以是：你的Java程序、JDK中某个类、Java运行时系统。

它们都是通过throw语句来实现的。

throw语句的格式为：**throw ExceptionInstance;**

4、必要时可以继承Exception或者其子类，定义自己的异常类。



2.3.2 try-catch

◆一个使用到所有关键字的方法框架，可能是这个样子:

```
void myMethod(<参数定义>) throws xxException
{ //xxException可以是自定义异常，也可以是Java提供的异常
    try {
        ……
        throw new ExceptionType(); //检测到异常状况可以主动抛出异常
        ……
    }
    catch (ExceptionType ex) {
        ……
        throw new MyException(); //必要的处理后可以再次抛出异常
    } finally  {
        //资源清理工作
    }
}
```

◆当异常被抛出时，流程发生跳转，到catch语句块执行。

◆无论是否有异常，方法结束前finally语句块总是被执行。

2.3.2 try-catch

- 假设想要一个程序来计算用户输入的实数序列的平均值，并且希望用户通过输入一个空行来表示序列的结束。
- 如果使用 input.nextLine()来读取用户的输入，使用 Double.parseDouble将非空白输入转换为数字。
- 在计算平均值的过程中，使用 try..catch 来避免当用户输入的不是合法数字时程序崩溃。

练习：在目前为止的例子代码中，nextInt, nextDouble都会因为用户输入不合法导致程序崩溃。试着修改这些代码，当输入不是合法的数字时，要求用户重新输入。

◆这是完整代码

```
import java.util.Scanner;
public class E0221ComputeAverage2
{
  public static void main(String[]
args) {
    String str; // 用户输入.
    double number; // 输出转换为数字.
    double total; // 总和.
    double avg; // 数字的平均值.
    int count; // 输入的数字数量.
    total = 0;
    count = 0;
    System.out.println("Enter your
numbers, press return to end.");
    Scanner input=new
Scanner(System.in);
    while (true) {
      System.out.print("? ");
      str = input.nextLine();
      if (str.equals("")) {
        break; // 退出循环
      }
      try {
        number =
Double.parseDouble(str);
      // 如果解析错误，后两行代码不执行
        total = total + number;
        count = count + 1;
      }
      catch (NumberFormatException e)
{
        System.out.println("Not a
legal number! Try again.");
      }
    }
    avg = total/count;
    System.out.printf("The average of
%d numbers is %1.6g%n", count, avg);
    input.close();
  }
}
```

◆例：参数检查错误，可以自己抛出IllegalArgumentException

```
public class IllegalArgumentExceptionExample {

    /**
     * @param parent, The path of the parent node.
     * @param filename, The filename of the current node.
     * @return The relative path to the current node, starting from the parent node.
     */
    public static String createRelativePath(String parent, String filename) {
        if(parent == null)
            throw new IllegalArgumentException("The parent path cannot be null!");
        if(filename == null)
            throw new IllegalArgumentException("The filename cannot be null!");
        return parent + File.separator + filename;
    }

    public static void main(String[] args) {
        // The following command will be successfully executed.
        System.out.println(IllegalArgumentExceptionExample.createRelativePath("dir1",
"file1"));
        System.out.println();
        // The following command throws an IllegalArgumentException.
        System.out.println(IllegalArgumentExceptionExample.createRelativePath(null,
"file1"));
    }}
```

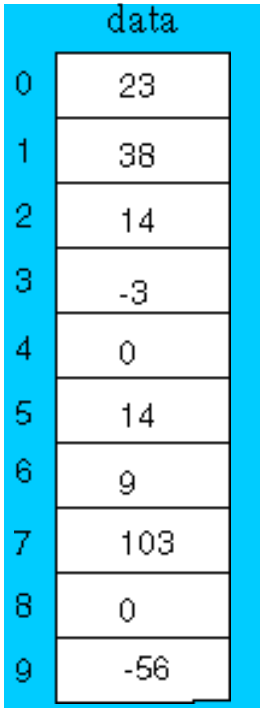
2.4 数组

◆ 数组由同一类型的一连串对象或者基本数据组成，并封装在同一个标识符（数组名称）下。

◆ 数组是对象

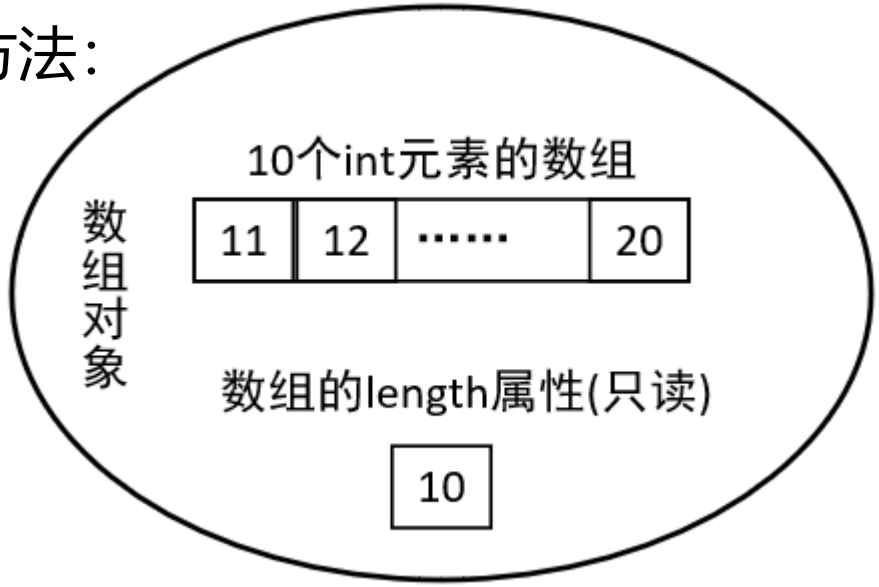
- ■动态初始化
- ■可以赋值给Object类型的变量
- ■在数组中可以调用类Object 的所有方法

◆ 数组中的各元素是有先后顺序的，它们在内存中按照这个先后顺序连续存放在一起；



◆数组元素

- ■数组中的变量被称作数组的元素
- ■元素没有名字，通过数组名字和非负整数下标值引用数组元素。
- ■每个数组都有一个由 public final 修饰的成员变量：*length*，即数组含有元素的个数（*length*可以是正数或零）
- ■注意：对比字符串长度是一个方法：
  - ➢String str = “Hello”；
  - ➢int len = str.length();



2.4.1 数组的创建和使用

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- 数组的声明
- 数组的创建
- 数组元素的初始化
- 数组的引用
- 多维数组

2.4.1 数组的创建和使用

- 数组的声明（Declaration）
  - 声明数组时无需指明数组元素的个数，也不为数组元素分配内存空间
  - 不能直接使用，必须经过初始化分配内存后才能使用

2.4.1 数组的创建和使用

- 数组的声明（Declaration）

Type[ ] arrayName;

例如：

int[] intArray;

String[] stringArray;

Type arrayName[ ];

例如：

int intArray[];

String stringArray[];

2.4.1 数组的创建和使用

- 数组的创建
  - 用关键字new构成数组的创建表达式，可以指定数组的类型和数组元素的个数。元素个数可以是常量也可以是变量
  - 基本类型数组的每个元素都是一个基本类型的变量；引用类型数组的每个元素都是对象的的引用
  - 创建后数组元素的值都是对应类型的默认值

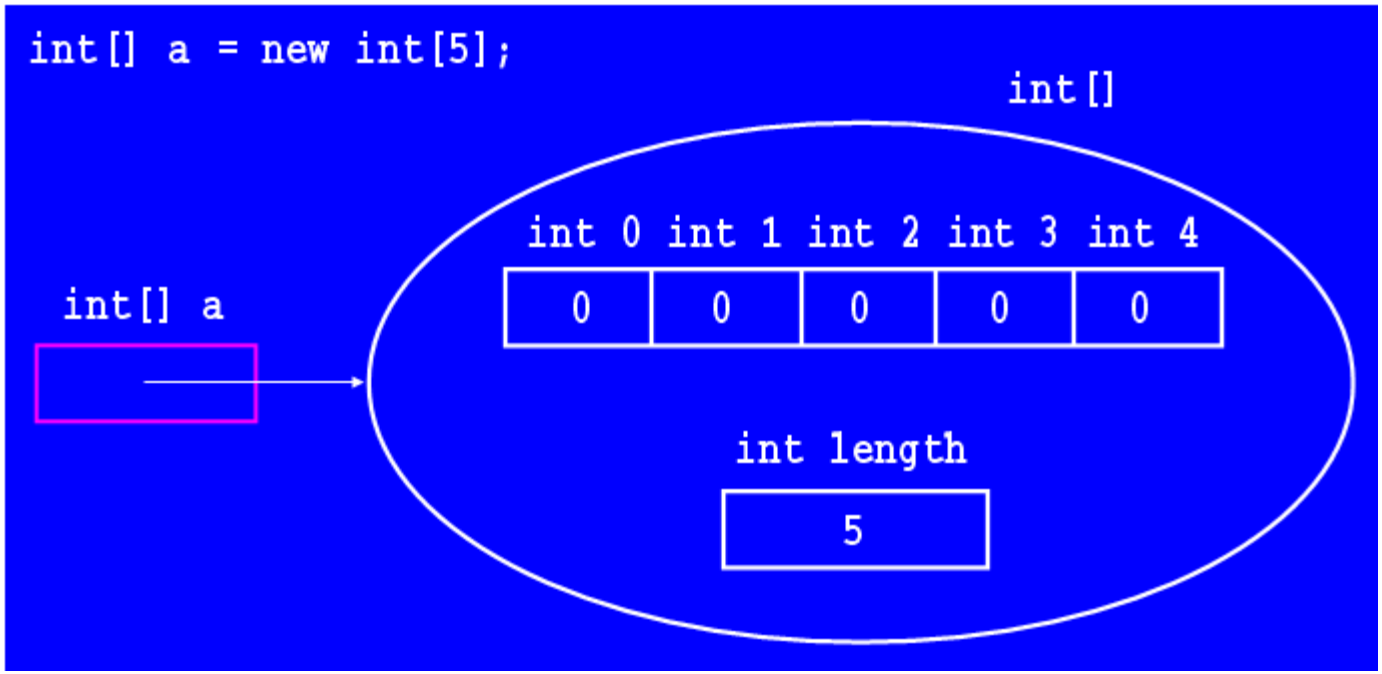
2.4.1 数组的创建和使用

arryName=new Type[componets number];

- 例如：
  int[] ai;　ai=new int[10];
  String[] s;　s=new String[3];
- 或者可以将数组的声明和创建一并执行
  int ai[]=new int[10];
- 可以在一条声明语句中创建多个数组
  String[] s1=new String[3], s2=new String[8];

2.4.1 数组的创建和使用

◆例如: 数组a用new创建后，length属性为5.每个元素都是0.



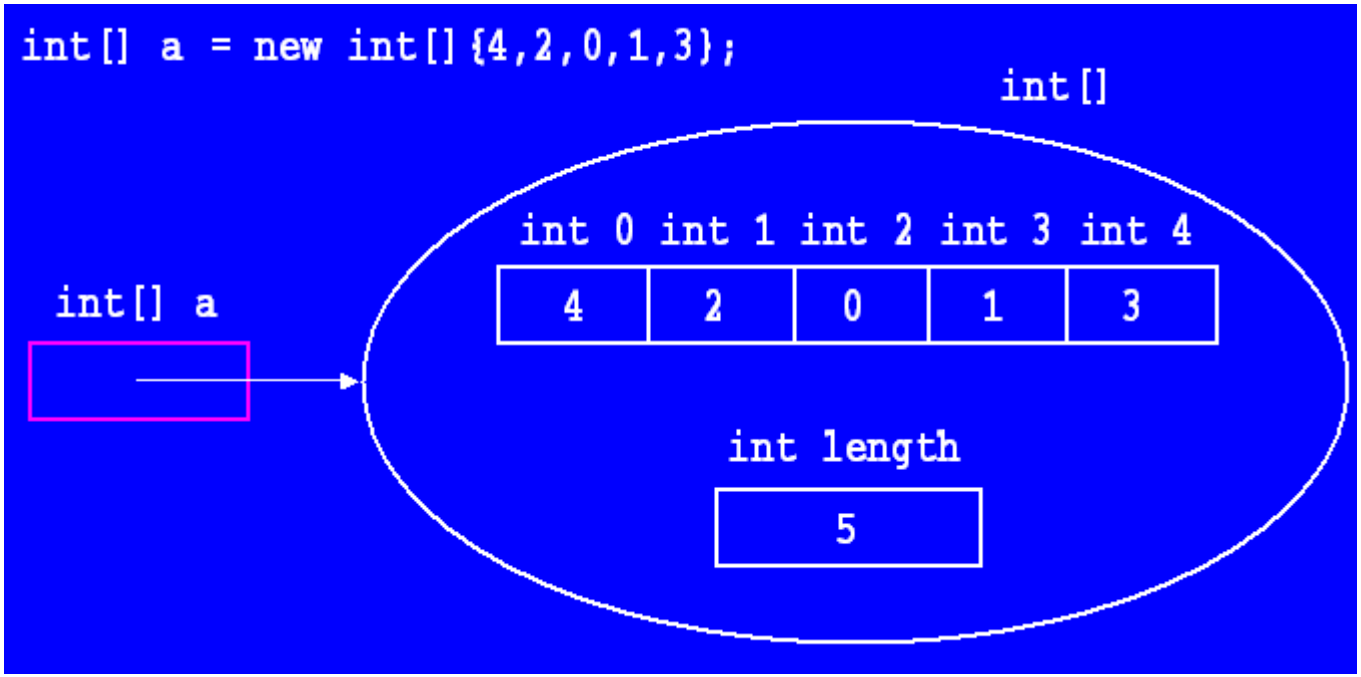
2.4.1 数组的创建和使用

- 数组元素的静态初始化
  - 声明数组名时，给出了数组的初始值，程序便会利用数组初始值创建数组并对它的各个元素进行初始化

    int a[]={22, 33, 44, 55};
- 创建数组的时，如果没有指定初始值，数组便被赋予默认值初始值。
  - 基本类型数值数据，默认的初始值为0；
  - boolean类型数据，默认值为false;
  - 引用类型元素的默认值为null。
- 程序也可以在数组被构造之后改变数组元素值

2.4.1 数组的创建和使用

- 例如：数组a利用静态初始化结果，length为5.



2.4.1 数组的创建和使用

- 数组的引用
  - 通过下面的表达式引用数组的一个元素：
    arrayName[index]
  - 数组下标必须是 int , short, byte, 或者 char.
  - 下标从零开始计数.
  - 元素的个数即为数组的长度，可以通过arryName.length引用
  - 元素下标最大值为 length – 1，如果超过最大值，将会产生数组越界异常（ArrayIndexOutOfBoundsException）

2.4.1 数组的创建和使用

- 数组的引用(续)

  int[] data = new int[10];
  - data[-1]　非法的
  - data[10]　非法的，越界
  - data[1.5]　非法的
  - data[0]　合法的
  - data[9]　合法的

2.4.1 数组的创建和使用

- 数组的引用(续)

```
int values[] = new int[7];
int index;
index  = 0;
values[ index ] = 71;
index  = 5;
values[ index ] = 23;
index  = 3;
values[ 2+2 ] = values[ index-3 ];
```

2.4.1 数组的创建和使用

- 数组的引用(续)

```
int values[] = new int[7];
int index;
index  = 0;
values[ index ] = 71;
index  = 5;
values[ index ] = 23;
index  = 3;
values[ 2+2 ] = values[ index-3 ];
```

2.4.1 数组的创建和使用

◆数组的引用(续)

```
public class E0222ArrayAccess {
    public static void main(String[] args){
        int myArray[];      //声明数组
        myArray=new int[10]; //创建数组,每个元素都是0.

        System.out.println("Index\t\tValue");
        //用传统的for循环遍历数组
        for(int i=0; i<myArray.length;i++)
            System.out.println(i+"\t\t"+myArray[i]);
        //用foreach遍历数组
        System.out.print("Values:");
        for(int i : myArray) System.out.print(i+" ");
            System.out.print("\n");
        //证明数组元素默认初始化为0
        //myArray[10]=100;  //将产生数组越界异常
    } }
```

2.4.1 数组的创建和使用

- 数组名是一个引用,比如下面代码，给a2每个元素赋值后，用a1输出的结果也随之改变。

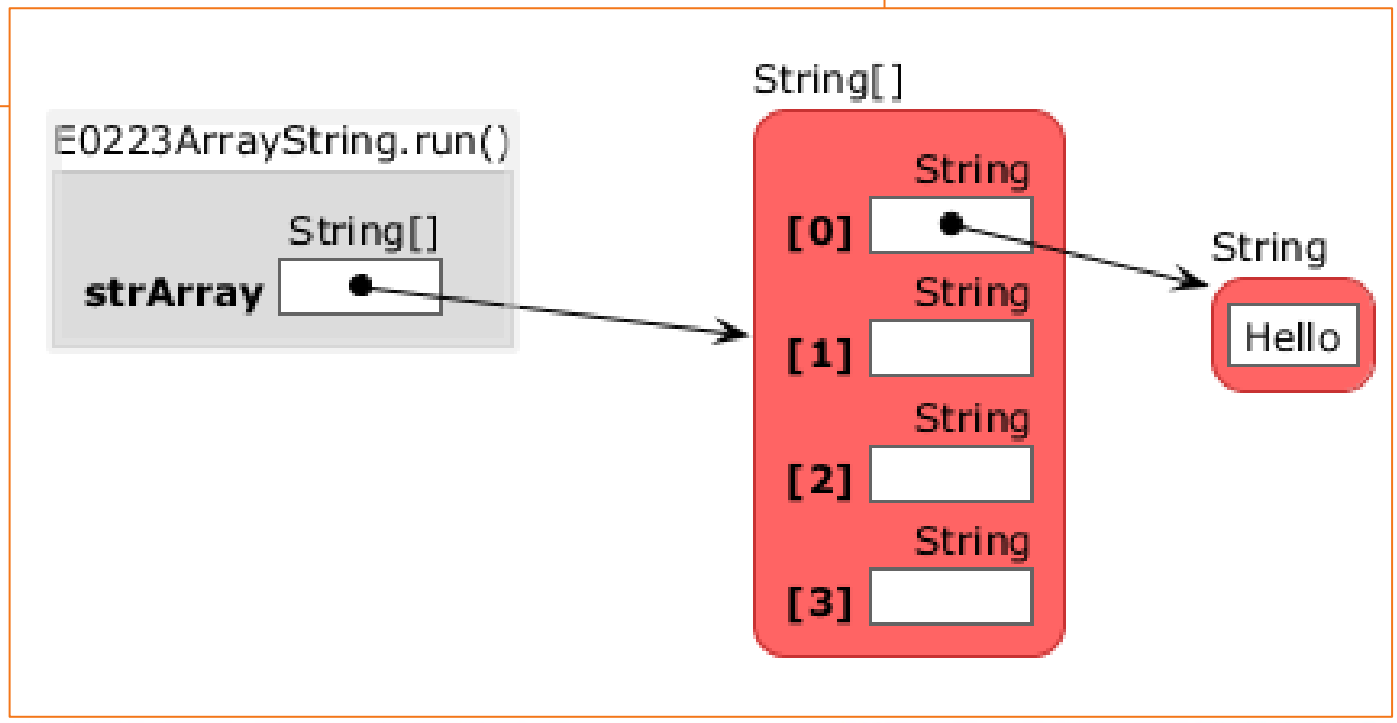
```
public class E0223ArrayNameIsReference
{   public static void main(String[] args)
    {  int[] a1 = { 1, 2, 3, 4, 5 };
        int[] a2;
        a2 = a1; //a2和a1指向同一个数组
        for(int i = 0; i < a2.length; i++)   a2[i]++;
        for(int i = 0; i < a1.length; i++)
            System.out.println( "a1[" + i + "] = " + a1[i]);
    }}
```

```
$java E0223ArrayNameIsReference.java
a1[0] = 2
a1[1] = 3
a1[2] = 4
a1[3] = 5
a1[4] = 6
```

2.4.1 数组的创建和使用

- 字符串引用构成的数组创建：

```
public class E0223ArrayString {
    void run() {
        String[] strArray;
        strArray = new String[4] ;
        strArray[0]= new String("Hello" );
    }
}
```



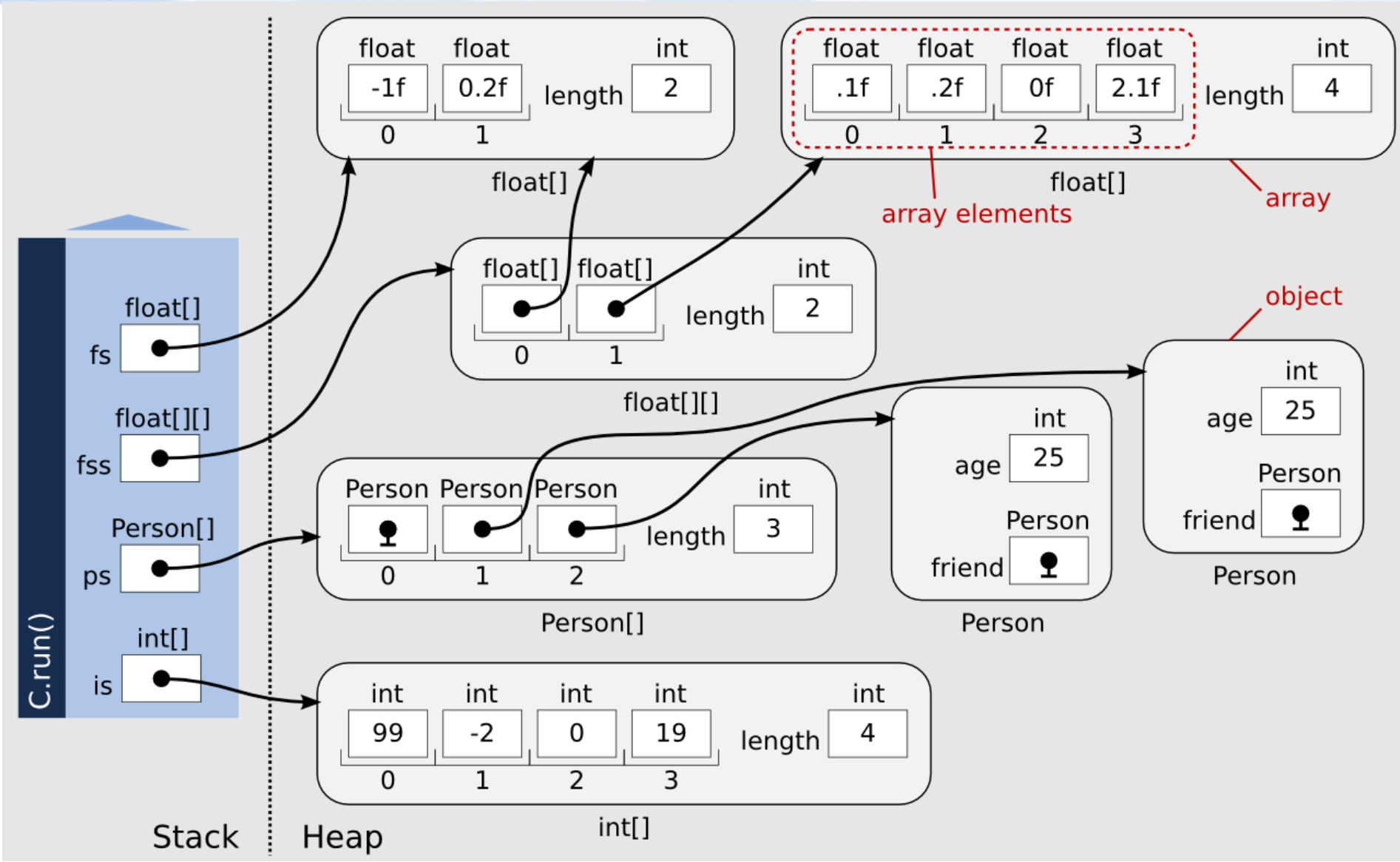
◆如下代码创建的变量和数组映像见下页

```
class Person {
    int age=25;
    Person friend;
}
public class E0223ArrayObjects {
  public static void main(String[]
args) {
    float[] fs = new float[2];
    fs[0]=-1.0f; fs[1]=0.2f;

    float[][] fss = new float[2][];
    fss[0]=fs;
    fss[1] = new float[4];
    fss[1][0]=0.1f; fss[1][1]=0.2f;
    fss[1][0]=0.f; fss[1][1]=2.1f;
    Person[] ps = new Person[3];
    ps[1]= new Person(); ps[2]=
new Person();

    int[] is = new int[4];
    is[0]= 99; is[1]=-2;
    is[1]= 0; is[2] =19;
  }
}
```

2.4.1 数组的创建和使用



2.4.2 数组和 for 循环

◆数组的许多真正强大之处在于元素的索引可以由整数变量甚至整数值表达式给出。

■假设 A 是一个 double 数组，求该数组所有元素的平均值。针对该问题，使用 for 循环将数字相加，然后除以数组的长度得到平均值：

```
double total; // 数组中元素的和.
double average; // 数组元素的平均值.
int i; // 数组元素的下标.
total = 0;
for ( i = 0; i < A.length; i++ ) {
    total = total + A[i]; // 求和.
}
average = total / A.length; // 计算平均值
```

◆查找数组 A 中的最大元素:

■可以通过遍历数组，将已经遍历过的数组元素的最大值存储在一个名为 max 的变量中，遍历完整个数组，max 的值就是要找的最大值。当遍历数组元素时，如果发现元素值大于当前 max 值，将 max 的值更改为元素的值，处理完整个数组后，max 是整个数组中最大的元素。

```
double max; //设置一个最大值变量.
max = A[0]; // 数组的第一个元素是 A[0].
for (int i = 1; i < A.length; i++ ) {
    if (A[i] > max) {//如果数组元素值大于当前的最大值 max
        max = A[i];
    }
}
```

- 如果程序只是处理数组的个别元素，可以在 for 循环中使用 if 语句来实现，通过 if 判定是否处理给定元素。
  - 例如，计算数组中非零元素的平均值。此时，相加的数组元素可能小于数组的长度，因此需要对求和的元素数量进行计数：

2.4.2 数组和 for 循环


```
double total; // 非 0 数组元素的和.
int count; // 非 0 数组元素的个数.
double average; // 非 0 元素的平均值.
total = 0;
count = 0;
for ( int i = 0; i < A.length; i++ ) {
    if ( A[i] != 0 ) {
        total = total + A[i]; // 求和
        count = count + 1; // 元素数量计数.
    }
}
if (count == 0) {
    System.out.println("There were no non-zero elements.");
}
else {
    average = total / count; // 计算平均值
    System.out.printf("Average of %d elements is
%1.5g%n",count, average);
}
```

2.4.3 随机访问数组

◆数组也可以实现随机访问，即，数组的每个元素在任何给定时间都可以被同等访问。

- 假设一个房间里有 N 个人，房间里有两个生日相同的人的可能性有多大？也就是说，他们出生在同一个月的同一天，但不一定是同一年。
- 随机选择一个人并检查他们的生日。在找到生日相同的人之前，可能会检查过很多人。当然，已经检查过的人的数量取决于随机因素，但可以用计算机程序模拟实验，并多次运行该程序，以了解平均需要检查多少人。
- 为了模拟实验，设置 365 个不同的可能生日（将忽略闰年）。对于每个可能的生日，需要跟踪是否已经找到过相同生日的人。为了保存所有 365 个可能生日的数据，使用一个包含365 个布尔值的数组：

```
boolean[] used;
used = new boolean[365];
```

2.4.3 随机访问数组

- 对于这个问题，一年中的日期从 0 到 364 编号。如果选择了生日为第 i 天的某人，则 used[i]的值为真。
- 最初，在创建数组时，让数组中所有值都是假的。这个过程可以让编译器自动完成的。当选择生日是第 i 天的人时，首先检查 used[i]是否为真。如果是真的，那么这是第二个过生日的人。另一方面，如果 used[i]为 false，将 used[i]设置为 true，以记录遇到过那个生日的人这一事实，然后继续寻找下一个人。
- 下面是一个进行模拟实验的程序（当然，程序中没有模拟人，只有模拟生日）：

2.4.3 随机访问数组

```
public class E0224BirthdayProblem {
    public static void main(String[] args) {
        boolean[] used; // 记录所有生日的数组
        // 如果发现 used[i]为真，表明已经存在一个人也是这一天的生日.
        int count; // 已经检查过的人员数量.
        used = new boolean[365]; // 初始化，所有单元为 false.
        count = 0;
        while (true) {
            // 在 0 到 364 范围选择一个生日.
            int birthday; // 选择一个生日.
            birthday = (int)(Math.random()*365);
            count++;
            System.out.printf("Person %d has birthday number %d%n", count,
birthday);
            if ( used[birthday] ) {
                //如果生日已经被使用，退出 .
                break;
            }
            used[birthday] = true;
        } // end while
        System.out.println();
        System.out.println("A duplicate birthday was found after "+ count +
" tries.");
    }
} // end class BirthdayProblem
```

2.4.4 二维数组

int[][] gradeTable;

……

gradeTable[0][1] 为42
gradeTable[3][4] 为93
gradeTable[6][2] 为78

Student	Week				
	0	1	2	3	4
0	99	42	74	83	100
1	90	91	72	88	95
2	88	61	74	89	96
3	61	89	82	98	93
4	93	73	75	78	99
5	50	65	92	87	94
6	43	98	78	56	99

2.4.4 二维数组

- 二维数组的声明和构造
  - int[ ][ ] myArray ;
    - myArray 可以存储一个指向2维整数数组的引用。其初始值为null。
  - int[ ][ ] myArray = new int[3][5] ;
    - 建立一个数组对象，把引用存储到myArray。这个数组所有元素的初始值为零。
  - int[ ][ ] myArray = { {8,1,2,2,9}, {1,9,4,0,3}, {0,3,0,0,7} };
    - 建立一个数组并为每一个元素赋值。

2.4.4 二维数组

- 二维数组的长度

```
class E0227UnevenArray{
    public static void main( String[ ] arg ) {
        int[ ][ ] uneven =
        { { 1, 9, 4 },
          { 0, 2},
          { 0, 1, 2, 3, 4 } };
        System.out.println("Length is: " +   uneven.length );
        System.out.println("Length uneven[0] is: "
+      uneven[0].length );
        System.out.println("Length uneven[1]is: "
+   uneven[1].length );
        System.out.println("Length uneven[2]is: "
+   uneven[2].length );
    }
}
```

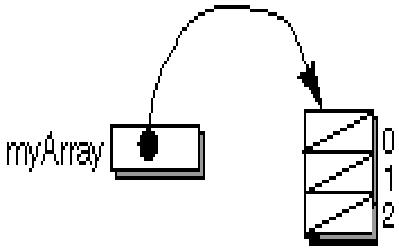
```
$java E0227UnevenArray.java
Length is: 3
Length uneven[0] is: 3
Length uneven[1]is: 2
Length uneven[2]is: 5
```

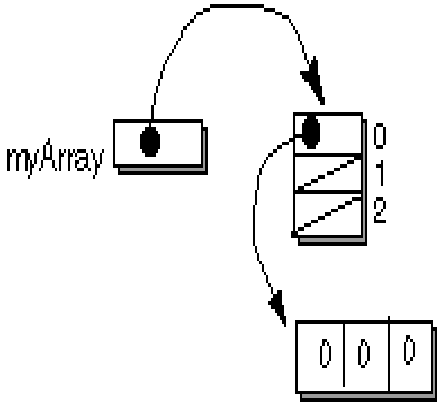
2.4.4 二维数组

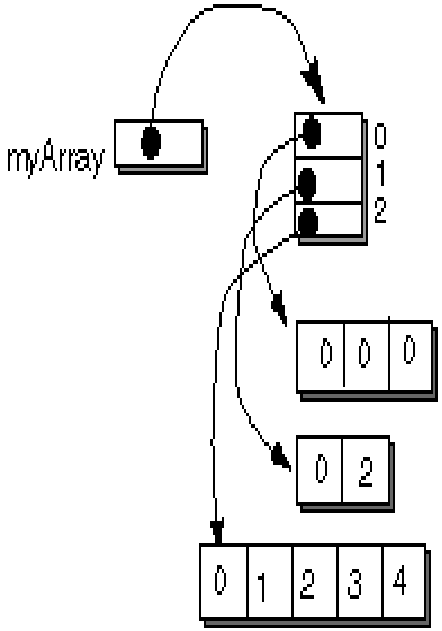
```
int[ ][ ] myArray;
myArray = new int[3][ ] ;
myArray[0] = new int[3];
```

```
int[ ] x = {0, 2};
int[ ] y = {0, 1, 2, 3, 4} ;
myArray[1] = x ;
myArray[2] = y ;
```









2.4.4 二维数组

◆考虑一个大型的超市，它有 25 家门店的，假设要存储2022 年中每个月每个商店赚取的利润时，就需要定义二维数组。其中第一维是门店的编号，0 到 24，第二维是 2022 年的月份，1 月到 12 月，编号从 0 到 11，那么，利润就可以存储在一个数组中，创建利润数组如下:

```
double[][] profit;
profit = new double[25][12]
```

◆对于 profit[storeNum][monthNum]，storeNum 代表门店编号，monthNum 代表月份。

◆假设利润数组已经输入，可以通过以下代码来计算公司 2022 年全年的利润:

2.4.4 二维数组

```
double totalProfit; // 公司 2022 年全年利润.
int store, month; // 定义门店号、月份变量
totalProfit = 0;
for ( store = 0; store < 25; store++ ) {
    for ( month = 0; month < 12; month++ )
        totalProfit += profit[store][month];
}
```

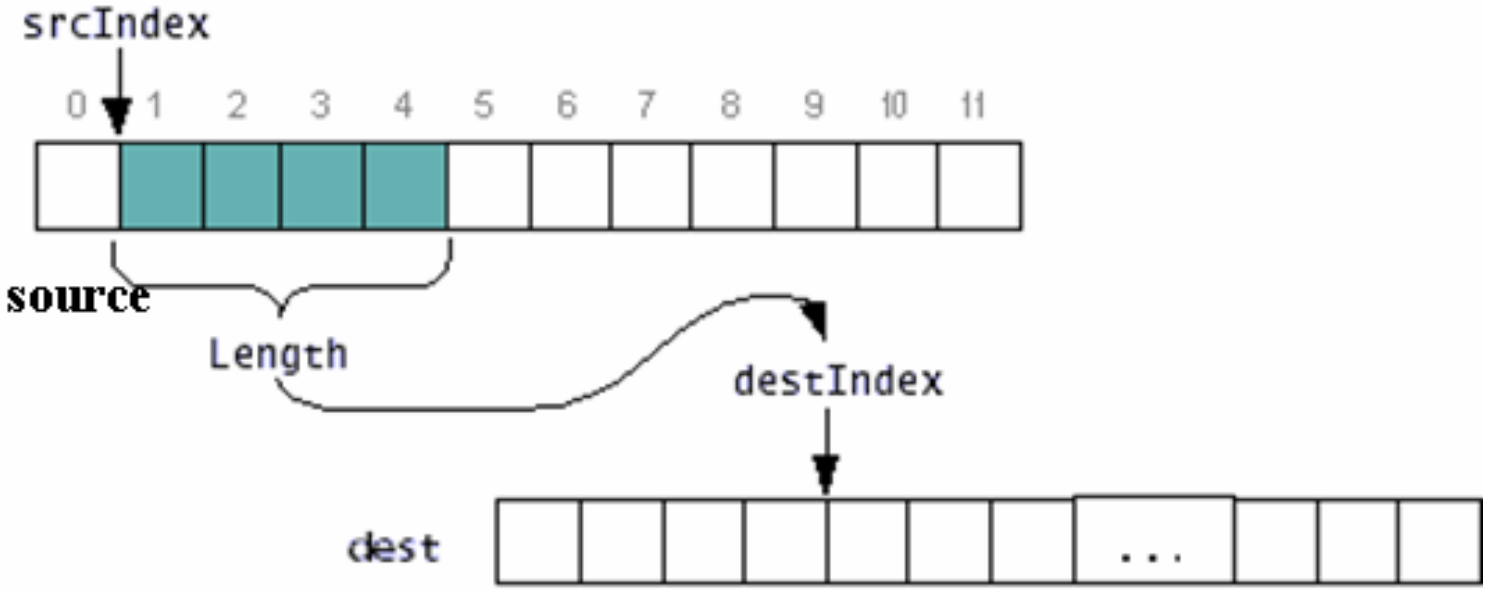
◆要计算公司在 12 月（即第11 个数组元素）赚取的总利润，可以使用以下循环（处理单列数据）

```
double decemberProfit; //计算12月利润
int storeNum;
decemberProfit = 0.0;
for ( storeNum = 0; storeNum < 25; storeNum++ ) {
    decemberProfit += profit[storeNum][11];
}
```

2.4.5 复制数组*

- 数组的复制： System类提供的方法

public static void arraycopy(Object *source* , int *srcIndex* , Object *dest* , int *destIndex* , int *length*)



2.4.5 复制数组*

例子

```
public class E0225ArrayCopyDemo
{ public static void main(String[] args)
  { char[] copyFrom = { 'd', 'e', 'c', 'a', 'f',
'f', 'e', 'i', 'n', 'a', 't', 'e', 'd'};
    char[] copyTo = new char[7];
    System.arraycopy(copyFrom, 2, copyTo, 0, 7);
    System.out.println(copyTo);//println可以直接显示
字符数组
   } }
```

```
$java E0225ArrayCopyDemo.java
caffein
```

2.4.5 复制数组*

◆注意：如果数组元素为引用类型，则复制就是所谓的“浅复制”(Shallow copy)。

- 对象拷贝时仅复制对象本身和对象中的基本数据类型字段的值，但不复制对象中引用类型字段所引用的对象。结果是，新对象中的引用类型字段实际上仍然指向原始对象中的同一个对象，而不是指向一个新的复制品。

◆举例: E2226ShallowCopy.java

2.4.5 复制数组*


```
class Car {
    int door;
}
public class E0226ShallowCopy {
    public static void main(String[] args){
        Car srcArray[];                          //声明数组
        srcArray=new Car[10];                    //创建数组

        //创建Car数组
        for(int i=0; i<srcArray.length;i++)
        {
            srcArray[i] = new Car();
            srcArray[i].door = i;
        }

        Car[] destArray = new Car[10];
```

2.4.5 复制数组*

```
        //这是浅拷贝
        System.arraycopy(srcArray, 0, destArray, 0,
destArray.length);
        srcArray[5].door = 50; //修改源数组对象，目标数组对象也随之改变

        //两个数组元素指向同一个对象,所以两个循环输出结果一样
        System.out.print("Values by  srcArray:");
        for(var car : srcArray)
            System.out.print(car.door+" ");
        System.out.println();

        System.out.print("Values by DestArray:");
        for(var car : destArray)
            System.out.print(car.door+" ");
        System.out.println();

        srcArray[5].door = 5; /*恢复到赋值前的状态*/
```

2.4.4 复制数组*


```
        /*这是深拷贝*/
        for (int i=0; i<srcArray.length; i++)
        {
            destArray[i] = new Car(); //对于引用变量重新创建实例
            destArray[i].door = srcArray[i].door;
        }
        srcArray[5].door = 50;
        System.out.println("重新拷贝的结果: ");
        /*两个循环输出的结果不一样*/
        System.out.print("Values by destArray:");
        for(var car : destArray)
            System.out.print(car.door+" ");
        System.out.println();

        System.out.print("Values by  srcArray:");
        for(var car : srcArray)
            System.out.print(car.door+" ");
        System.out.println();
    }}
```

◆由于含有汉字，文件编码是UTF-8,编译时指定UTF-8:

```
$javac -encoding UTF-8 E0226ShallowCopy.java

$java E0226ShallowCopy
Values by  srcArray:0 1 2 3 4 50 6 7 8 9
Values by DestArray:0 1 2 3 4 50 6 7 8 9
重新拷贝的结果:
Values by destArray:0 1 2 3 4 5 6 7 8 9
Values by  srcArray:0 1 2 3 4 50 6 7 8 9
```